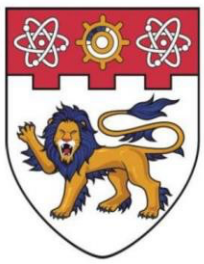
CLAHE HLS Implementation on Zynq FPGA *(B2100-231)**

Yoong Khai Jie, Nathaniel

Year
2023/2024

CLAHE HLS Implementation on Zynq SoC FPGA



NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

SCHOOL OF ELECTRICAL

AND ELECTRONIC ENGINEERING

Yoong Khai Jie, Nathaniel

Year 2023/2024

Project No: B2100-231

ABSTRACT

Infrared (IR) imaging is pivotal for military applications, facilitating detection in low-light scenarios. However, conventional IR images often suffer from contrast issues, particularly when displaying high and low-temperature objects in the same scene. This leads to a loss of contrast in the whole image. Contrast Limiting Adaptive Histogram Equalization (CLAHE) emerges as a solution, enhancing image quality by segmenting the image into tiles and applying histogram equalization techniques. This paper proposes the implementation of CLAHE on a Field Programmable Gate Array (FPGA), leveraging its real-time data processing capabilities. By employing High-Level Synthesis (HLS), the algorithm's execution on FPGA promises significant contrast enhancement, surpassing conventional methods like histogram equalization. The envisioned FPGA-based CLAHE system aims to offer superior IR image contrast crucial for military surveillance and reconnaissance applications

Project No: B2100-231

ACKNOWLEDGEMENTS

I would like to thank Mr Huang Wenhao Bryan, my supervisor from DSO National Laboratories for his constant support and guidance on the project. I would like to also thank my FYP supervisor Prof Radhakrishnan K for willingly taking me under his wing to be mentored for this project. Lastly, I am grateful to my examiner Prof Xiao Gaoxi for taking the time to assess and give his valuable feedback on this project. With their support, the project could meet its objectives and be completed.

TABLE OF CONTENTS

Abstract i

Acknowledgements ii

List of Figures vi

List of Tables ix

List of Symbols ..... **Error! Bookmark not defined.**

Chapter 1: Introduction 1

1.1 Background 1

1.2 Motivation 1

1.3 Objective & Scope 2

Chapter 2: Literature Review 3

2.1 CLAHE Concept 3

2.1.1 Transformation function 4

2.1.2 Linear Interpolation 4

2.1.3 Bilinear Interpolation 5

2.2 Xilinx CLAHE HLS Implementation 6

2.3 Improved Design and Implementation of CLAHE 7

2.3.1 DDR Memory 7

2.3.2 Floating Point Division Timing 7

2.3.3 Floating-Point IP Core 8

2.3.4 Axi4-Stream Protocol 8

Chapter 3: Hardware Implementation 10

3.1 FPGA Hardware 10

3.2 Supporting Applications 11

3.2.1 Jupyter Notebook 11

3.2.2 Tera Term 12
3.3 Vivado HLS 13
3.4 CDF Calculation IP Core 14
3.4.1 Initializing Function and Ports 15
3.4.2 Histogram Generation 16
3.4.3 CDF Generation 16
3.4.4 Minimum and Maximum Bin 18
3.4.5 Minimum CDF 19
3.4.5 DDR Memory Transactions 19
3.5 Bilinear Calculation IP Core 20
3.5.1 Initializing Function and Ports 20
3.5.2 Reading in CDF and Miscellaneous Values 22
3.5.3 Corner Cases 24
3.5.4 Edge Cases 25
3.5.5 Center Cases 27
3.5.6 Axi4-Stream Output 28
3.6 Histogram Equalization IP Core 29
3.7 Vivado IDE 31
3.7.1 CDF Block 31
3.7.2 Bilinear Block 32
3.7.3 Floating-Point IP Core 32
3.7.4 Histogram Equalization Block 34
3.7.5 Overall Block Design 35
3.8 Xilinx SDK 36
3.8.1 Initializing and Set up 36
3.8.2 Sequence of Starting 38

3.8.3 Timing Calculation and Printing Values 39
3.8.4 Programming FPGA 40
Chapter 4: Results 42
4.1 CLAHE-Enhanced Images 42
4.2 Throughput on Hardware 44
4.3 Resource Utilization 45
Chapter 5: Challenges Faced 46
5.1 CLAHE Algorithm Implementation 46
5.2 Meeting Throughput Requirements 47
5.3 Workings of Floating-Point IP Core 48
5.4 Resource Management in Bilinear HLS Core 48
Chapter 6: Future Work 50
6.1 Real-Time Operations 50
Chapter 7: Conclusion 50
7.1 Requirements 50
7.2 Findings Based on Results 51
7.3 Scalable Implementation 51
References 56

LIST OF FIGURES

Fig. 1.Representation of tiles, corner, edge and center cases in a general image.....4

Fig. 2.Representation of tiles, corner, edge and center cases in a general image.....6

Fig. 3. Chip scope view of time taken to execute floating-point division in floating-point IP core.....8

Fig. 4. Hardware of the FPGA board, USB-UART port (orange) and USB-JTAG port (blue).....10

Fig. 5. Diagram of project set-up.....10

Fig. 6. Code for adding commas after every line and removing white spaces in Python on Jupyter Notebook.....11

Fig. 7. Code for visualizing image from the raw pixel text file.....11

Fig. 8. Connecting the FPGA to the Laptop via Tera Term, configuring baud rate to 115200.....12

Fig. 9. Steps to enable logging function on Tera Term.....12

Fig. 10. Example of coding on Vivado HLS.....13

Fig. 11. Resource Utilization and timing analysis of HLS code.....14

Fig. 12. Initialization of function and ports in HLS CDF core.....15

Fig. 13. BINS initialized in #define.....16

Fig. 14. Code on clipping the histogram.....16

Fig. 15. Code on calculating and storing CDF values.....18

Fig. 16. Code on obtaining minimum and maximum bin values.....18

Fig. 17. Code on obtaining minimum CDF values.....19

Fig. 18. Code on transacting CDF values to output array with indexing.....19

Fig. 19. Code on transacting the miscellaneous data to DDR memory.....20

Fig. 20. Function of the Bilinear HLS Core.....21

Fig. 21. #pragma initialization on Bilinear Core. 22

Fig. 22. Code for reading in miscellaneous data and CDF values. 23

Fig. 23. Code of corner case and transformation function calculation. 24

Fig. 24. Code of edge cases parameter setting for x and y direction. 25

Fig. 25. Code of linear interpolation in the x direction. 26

Fig. 26. Code of linear interpolation in the y direction. 26

Fig. 27. Code of Bilinear interpolation parameter settings. 27

Fig. 28. Code of Bilinear interpolation equations. 27

Fig. 29. Code of Axi4-Stream value assigning and transmission. 28

Fig. 30. Code of Power function for HE Block. 29

Fig. 31. Code of function for HE Block.....30

Fig. 32. Code of quotient addition and shifting decimal place for HE Block. 30

Fig. 33. HLS IP Core of CDF calculation. 31

Fig. 34. HLS IP Core of Bilinear calculation block. 32

Fig. 35. Fixed-to-Float function in floating-point IP Core block. 33

Fig. 36. Divide function in floating-point IP Core block. 33

Fig. 37. Float-to-Fixed function in floating-point IP Core block. 33

Fig. 38. Histogram Equalization Block.....34

Fig. 39. Overall Block Design of the CLAHE Implementation. 35

Fig. 40. Initializing three HLS Blocks and Software Timer. 37

Fig. 41. Setting up parameters for CDF Block. 37

Fig. 42. Setting up parameters for HE and Bilinear Block. 37

Fig. 43. Starting sequence of CDF block and timing capture. 38

Fig. 44. Starting sequence of HE and Bilinear Block and timing capture. 39

Fig. 45. Fps calculation in Xilinx SDK.....39

Fig. 46. Printing the CLAHE-enhanced pixel values to Tera Term. 40

Fig. 47. Steps to Program FPGA. 40

Fig. 48. Steps to launch code on hardware. 41

Fig. 49. Original Image from the raw pixels given.....42

Fig. 50. Image from 4-tile CLAHE variant.....43

Fig. 51. Image from 12-tile CLAHE variant.....43

Fig. 52. Image from 16-tile CLAHE variant.....44

Fig. 53. Throughput results for 4, 12 and 15-tile CLAHE variant for CDF, Bilinear and HE Block. 45

Fig. 54. Resource Utilization for entire CLAHE hardware design. 46

Fig. 55. Erroneous 4-tile CLAHE-enhanced image on Xilinx SDK.....47

Fig. 56. Fps of Old Bilinear HLS Core Design.....48

LIST OF TABLES

Table 1 AXI4-STREAM PARAMETERS 9

CHAPTER 1: INTRODUCTION

1.1 Background

Infrared (IR) imaging has been used in the military for years to detect humans or objects in low-light conditions. These images are monochrome, and the image's brightness depends on the object's heat. For instance, a very hot object will appear white in the image and flare up the contrast of the image, making it difficult to see other features and objects. Contrast Limiting Adaptive Histogram Equalization (CLAHE) is an image enhancement algorithm that produces even more precise and defined images. This technique splits the image into non-overlapping segments of almost equal sizes called tiles. A histogram equalization technique is applied to each pixel in each tile. Bilinear interpolation is applied to each pixel with the pixels in the neighboring tiles to even out the contrast of the pixels, ensuring that all the details in the image can be seen in greater detail compared to the original image [1].

This algorithm will be implemented on a Field Programmable Gate Array (FPGA), a programmable logic device based around a matrix of configurable logic blocks connected by programmable interconnect [2]. It is mainly used in systems requiring high rates of real-time data transfer. One example is a system used for real-time imaging. Using the FPGA will potentially allow for real-time imaging of IR using CLAHE with more significant contrast enhancement relative to its predecessors, for example, ~~adaptive~~ histogram equalization [3]. The algorithm will be implemented in High Level Synthesis (HLS), a tool that allows users to write C code to be synthesized into VHDL/Verilog code.

1.2 Motivation

Existing solutions developed by Xilinx work for 8-bit input to 8-bit output [4]. However, IR Imaging generates 14-bit data, and thus, the existing Xilinx algorithm cannot be used. Hence, there is a need to self-develop a more comprehensive CLAHE algorithm that accommodates 14-bit input to 8-bit output. By self-developing the algorithm, its features will be more tailored to IR imaging applications.

1.3 Objective & Scope

The project aims to implement the CLAHE algorithm that can process the data at least 30 frames per second (fps) on the Xilinx Ultrascale+ MPSoC UltraZed-EG FPGA using HLS for contrast enhancement on 14-bit infrared images and generate 8-bit output images. No hardware description language code will be coded. The algorithm will be tested by dividing a 640 x 480-pixel image into 4, 12, or 16 tiles, with no clip limit for standardization. A serial monitor called Tera Term will log the output array into a log file. Jupyter Notebook, a Python code environment, will generate the image using the same log file generated using Python Code. Vivado and Vivado HLS will be needed for the Xilinx FPGA platform and will be run on Windows 10. In addition, the project will not cover real-time applications and will focus more on the performance and resource utilization of the FGPA. Instead, this array of 8-bit data will be stored in the Double Data Rate (DDR) Memory on board the FPGA.

CHAPTER 2: LITERATURE REVIEW

2.1 CLAHE Concept

The CLAHE algorithm addresses the known issue of amplification of unwanted noise in a general histogram equalization algorithm. CLAHE introduces a clip limit to overcome the issue of unwanted noise and tiles for improving local contrast of the neighboring tiles instead of global equalization of the entire image. CLAHE first splits the image into smaller equal segments called tiles. The 4-tile variation was split 2x2 ways, 12-tile 4x3 ways and 16-tile 4x4 ways. A histogram of the intensity of the pixels is generated for every tile. The histogram represents the number of pixels for each intensity value for every tile. For a 14-bit input, the range of the pixel values can theoretically range from 0 to 16383 ($2^{14} - 1$). However, it should be noted that the pixel intensity for IR images practically do not spread throughout the entire 14-bit range. To prevent the overamplification of unwanted noise, the clip limit clips the histogram at a predefined value. Cumulative distribution functions (CDF) will be generated for every tile from the histograms. The calculation for the resultant CLAHE image is dependent on the CDF generated for each tile.

In an image, there are three cases needing three different types of calculation. The corner case (in red in Fig. 1) uses only the transformation and histogram equalization (HE) function to calculate the resultant pixel value. The edge cases (green in Fig. 1), uses linear interpolation of HE of the neighboring tile. Lastly, the center case (blue in Fig. 1), uses bilinear interpolation of HE of neighboring tiles. The result will be an array of the resultant pixel values that would form the CLAHE image.

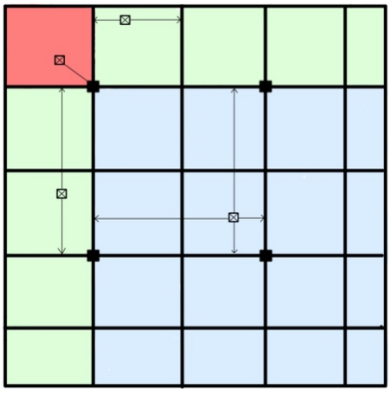


Fig. 1. Representation of tiles, corner, edge and center cases in a general image

2.1.1 Transformation function

The transformation function weighs the current CDF of a pixel value with the entire tile. As shown in the transformation function equation below, cdf_{min} is the minimum non-zero value of the CDF for that tile. M and N represents the width and height of one tile respectively, multiplying it gives one tile size. L represents the number of gray levels used. As our output is in 8-bit, L = $2^8 = 256$. This is the formula used to calculate the corner cases in an image.

$$f(x,y) = round\left(\frac{cdf(x,y) - cdf_{min}}{(M*N) - cdf_{min}} * (L-1)\right)$$

2.1.2 Linear Interpolation

Linear interpolation is used for the edge cases in the image. This includes the interpolation of the x and y direction of the green segments in Fig. 1. The equation used for calculating the CLAHE pixel for the x direction is where $Q_{11}, Q_{12}, Q_{21}, Q_{22}$ represents the transformation function for the CDF value in its own tile:

$$f(x,y) = \frac{(x_2-x)}{(x_2-x_1)}.\ Q_{11} + \frac{(x-x_1)}{(x_2-x_1)}.\ Q_{21}$$

Linear interpolation formula for the y axis is:

$$f(x,y) = \frac{(y_2-y)}{(y_2-y_1)}.\ Q_{11} + \frac{(y-y_1)}{(y_2-y_1)}.\ Q_{12}$$

There exists one equation for each direction as the neighboring tile in each direction is different and the current pixel relative to the anchor point would be different as well.

2.1.3 Bilinear Interpolation

Bilinear interpolation is used for calculating the center portion of the image as it helps to smoothen the intensity transitions between neighboring tiles, resulting in a smoother overall appearance in the final enhanced image. The equation below shows the calculation of how the CLAHE pixel is generated for the center portion. $Q_{11}, Q_{12}, Q_{21}, Q_{22}$ represents the transformation function for the CDF value in its own tile. Bilinear interpolation weighs the coordinates of the current pixel relative to predefined anchor points, x_1, x_2, y_1, y_2.

$$f(x,y) = \frac{(y_2-y)(x_2-x)}{(y_2-y_1)(x_2-x_1)}.\ Q_{11} + \frac{(y_2-y)(x-x_1)}{(y_2-y_1)(x_2-x_1)}.\ Q_{21}$$
$$+ \frac{(y-y_1)(x_2-x)}{(y_2-y_1)(x_2-x_1)}.\ Q_{12} + \frac{(y-y_1)(x-x_1)}{(y_2-y_1)(x_2-x_1)}.\ Q_{22}$$

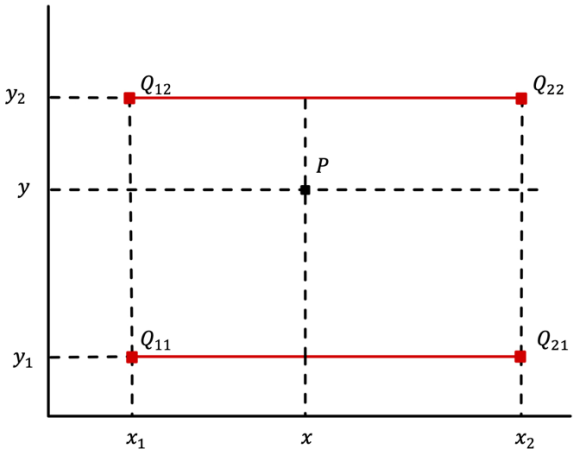


Fig. 2. Representation of tiles, corner, edge and center cases in a general image

2.2 Xilinx CLAHE HLS Implementation

The current Xilinx CLAHE HLS implementation only accepts an 8-bit input and outputs 8-bit. This is because the CDFs are stored internally in the BRAMs of the FPGA that are limited in memory storage [5]. As the CDF memory storage increases exponentially with pixel width 2^n, n being the number of bits per pixel, another solution to implement CLAHE on the FPGA is needed.

2.3 Improved Design and Implementation of CLAHE

As raw images from the IR camera are in 14-bit monochrome, the algorithm must be flexible to take in and enhance the local contrast of 14-bit IR images and output 8-bit monochrome image for downstream computer display. It must also cater to custom image size and number of tiles. In addition, the algorithm must address the issue of limited internal FPGA memory storage for the CDFs.

2.3.1 DDR Memory

One improvement implemented was to store the CDF values for each tile in DDR memory, as the DDR memory on board the FPGA is much larger than the BRAM. This change immediately solves the limitation of insufficient memory space for the CDFs. The external DDR memory size on the FPGA is 2GB which is more than enough to store the CDF values.

2.3.2 Floating-Point Division Timing

The first iteration of the project which involved the floating-point divisions of the resultant CLAHE pixels being done inside the HLS core did not meet the throughput requirements as shown in Fig. 3. Upon further investigation, the iteration interval (throughput) for a floating-point division to be completed was 7 clock cycles (cc). This was detrimental as in a 16-tile implementation, there would be up to 921600 floating point divisions executed. The cumulative time for all these divisions would severely affect the throughput of the algorithm.

2.3.3 Floating-Point IP Core

To ensure the design meets the throughput requirement, experimentation on a Xilinx external floating-point intellectual property (IP) core was done as it could give the quotient faster than 9cc. The floating-point IP core has multiple function, of which divide, float to fixed number, and fixed to float number function were used in the design. As an experiment, custom single inputs were fed to the floating-point IP core blocks by General Purpose Input Output (GPIO) blocks. Another GPIO block receives the output and is read. It can be seen in Fig. 3 that the floating-point IP core manages to execute floating point division in 4cc where 1cc = 10ns, which is an improvement compared to floating point division in the HLS core and as such is used in the design. To be used in the design, the HLS core that handles the calculations must output the numerator and denominator of the equation separately to be fed into the floating-point IP core via Axi4-Stream.

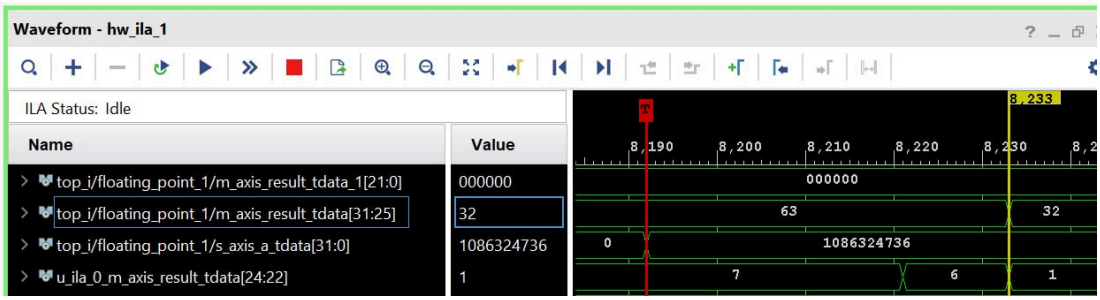


Fig. 3. Chip scope view of time taken to execute floating-point division in floating-point IP core.

2.3.4 Axi4-Stream Protocol

As the floating-point IP core transacts data using the Axi4-Stream Protocol, Axi4-Stream had to be implemented in the design. Axi4-Stream is a protocol designed for transporting arbitrary unidirectional data [6]. Important parameters used are TDATA, TVALID, TREADY, TUSER and TLAST.

TABLE 1 AXI4-STREAM PARAMETERS

Parameter	Usage	Bits
TDATA	Width of information bits transferred	64-bit width used
TVALID	Indicates valid data to be read by the receiver	'1': Data on the bus is valid, can be processed by the receiver '0': Receiver ignores data on TDATA bus
TREADY	Backpressure from the receiver to indicate to sender if the receiver is ready to accept data	'1': Receiver prepared to receive data '0': Receiver temporarily unable to accept new data
TUSER	Reserved for carrying auxiliary information	20-bit width used to indicate current pixel number being processed
TLAST	Indicates end of packet	'1': End of packet '0': Otherwise

CHAPTER 3: HARDWARE IMPLEMENTATION

3.1 FPGA Hardware

The FPGA used is the Xilinx UltraZed-EG SOM, which additionally consists of a carrier board. Features used on the board are the Universal Serial Bus– Universal Asynchronous Receive Transmit (USB - UART) port and the USB – Joint Test Action Group (USB - JTAG) port to connect to the Windows Laptop. For this project, the boot mode "Boot from JTAG" was selected for ease of implementation and efficient testing.

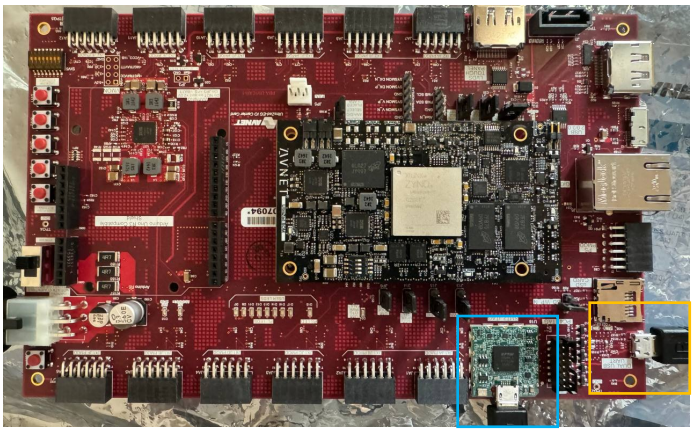


Fig. 4. Hardware of the FPGA board, USB-UART port (orange) and USB-JTAG port (blue).

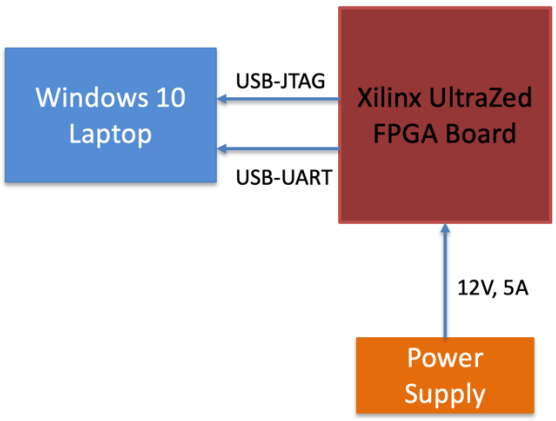


Fig. 5. Diagram of project set-up.

3.2 Supporting Applications

Other applications on the Windows laptop had to run to capture the data and visualize the image. For this project, Jupyter Notebook and Tera Term were used.

3.2.1 Jupyter Notebook

Jupyter Notebook is an open-sourced web application where users can run their code. For this project, Python was used to reformat and visualize the data on Jupyter Notebook. Jupyter Notebook was used to add commas and remove white spaces to the raw data text file such that it is easy to make into an array as shown in Fig. 5. In Fig. 6, Jupyter Notebook is used to visualize the image from the raw pixel text file.

```
In [7]:
import matplotlib.pyplot as plt
import numpy as np

with open('C:/Users/natyo/Desktop/CAP5-8_Raw/Test/CAP5.txt',"r") as input_file:
    with open('C:/Users/natyo/Desktop/CAP5-8_Raw/Test/CAP5_1.txt', "w") as output_file:
        for line in input_file:
            line = line.rstrip()  # remove any whitespace at the end of the line
            line_with_comma = line + ','  # add a comma at the end of the line
            output_file.write(line_with_comma + '\n')  # write the modified line to the output file
```

Fig. 6. Code for adding commas after every line and removing white spaces in Python on Jupyter Notebook.

```
In [8]:
import matplotlib.pyplot as plt
import numpy as np
# C:\Users\YKhaiJie\Desktop\TT_log_file
# C:/Users/YKhaiJie/Desktop/CAP5_8_Raw/CAP5.txt
with open("C:/Users/natyo/Desktop/CAP5-8_Raw/Test/CAP5.txt") as f:
    mylist = f.read().splitlines()

pixel_values = np.array(mylist, dtype=np.uint16)
image_data = pixel_values.reshape(480,640)
plt.imshow(image_data, cmap='gray')
plt.show()
```

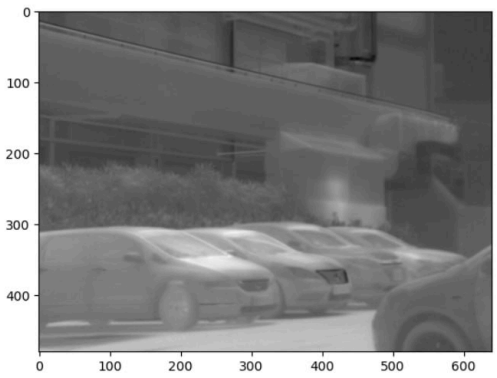


Fig. 7. Code for visualizing image from the raw pixel text file.

3.2.2 Tera Term

Tera Term is a free open-sourced serial terminal that logs and displays the CLAHE pixels. The baud rate setting is set to match the baud rate of the ARM processor on the FPGA at 115200 bps. The setting up of Tera Term and logging of the data printed is shown in Fig. 7 and Fig. 8.

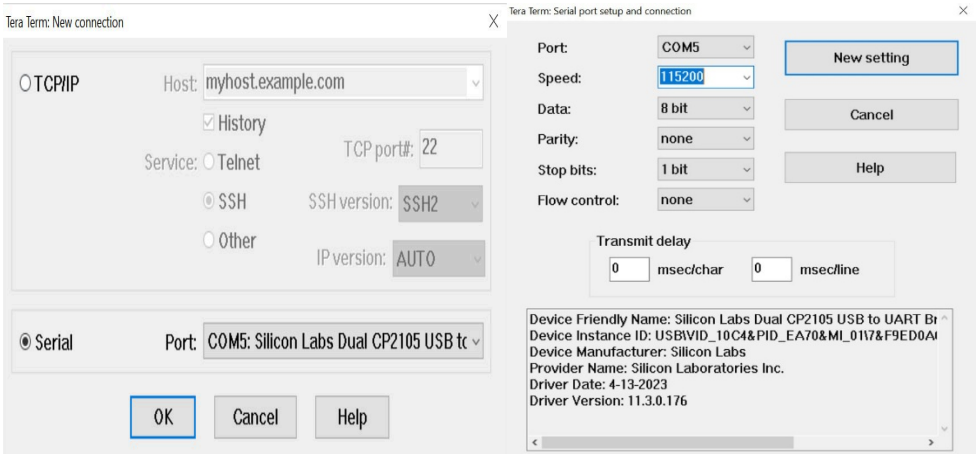


Fig. 8. Connecting the FPGA to the Laptop via Tera Term, configuring baud rate to 115200.

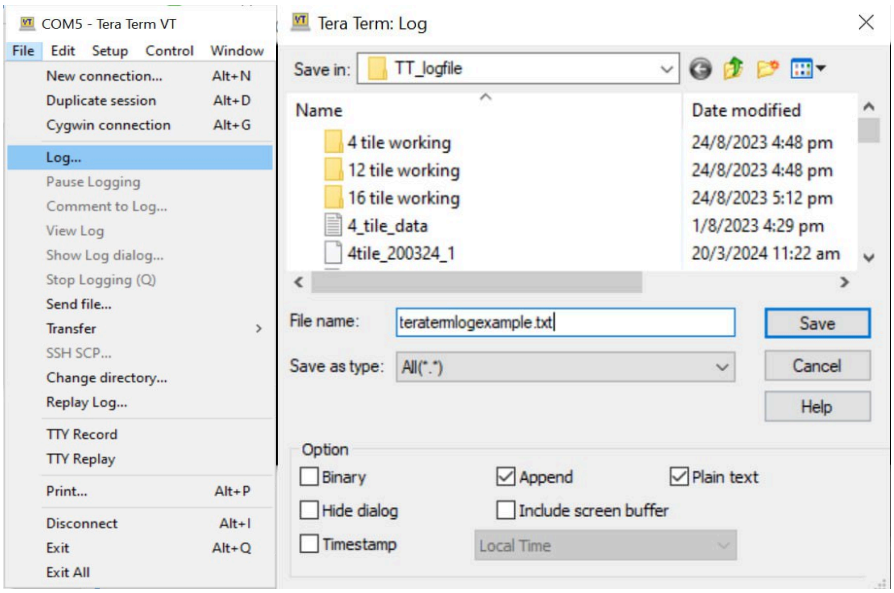


Fig. 9. Steps to enable logging function on Tera Term.

3.3 Vivado HLS

Vivado HLS is a tool Xilinx provides to create hardware designs in C instead of the traditional hardware description languages (HDLs) such as Verilog or VHDL. The primary use of Vivado HLS in this project is for faster development and easier verification compared to using HDL. Vivado HLS was used to design and create 3 HLS IP Cores, the CDF calculation core, the Bilinear calculation core, and the Histogram Equalization (HE) Core.

In Vivado HLS, the C code is simulated, synthesized and exported as an IP Block. Fig. 10 is an example of the Vivado HLS workflow. In the HLS design, the user can configure specific parameters in Xilinx SDK before the code is run on the FPGA. Some parameters to be configured before running the program are the number of tiles in the x and y direction, tile size, image width and height and clip limit. After synthesizing the HLS code, a report on resource utilization and an analysis of timing will be given. The HLS code was streamlined to ensure that both resource utilization and timings were of acceptable numbers. Fig. 11 shows the resource utilization report and the timing analysis given by HLS.

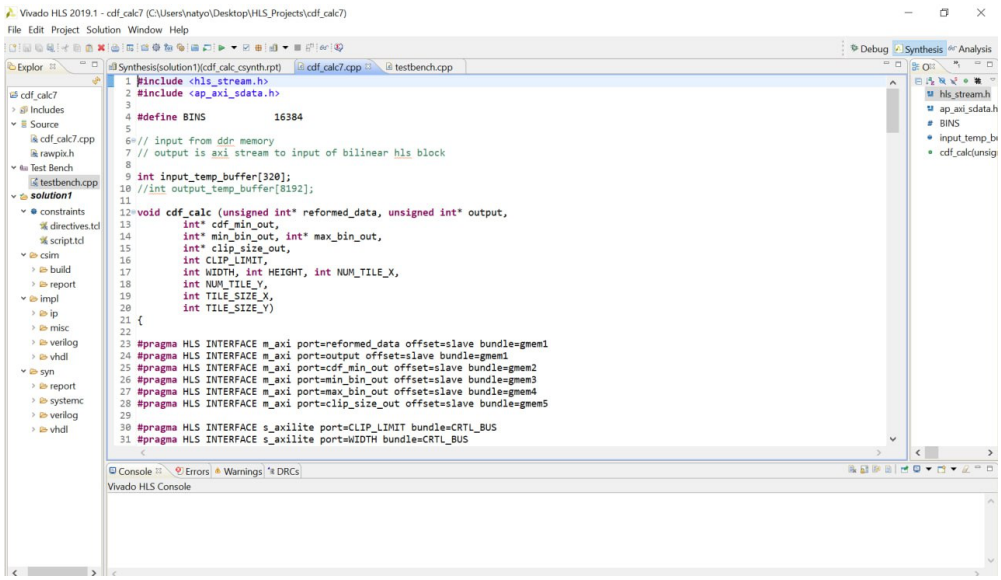


Fig. 10. Example of coding on Vivado HLS.

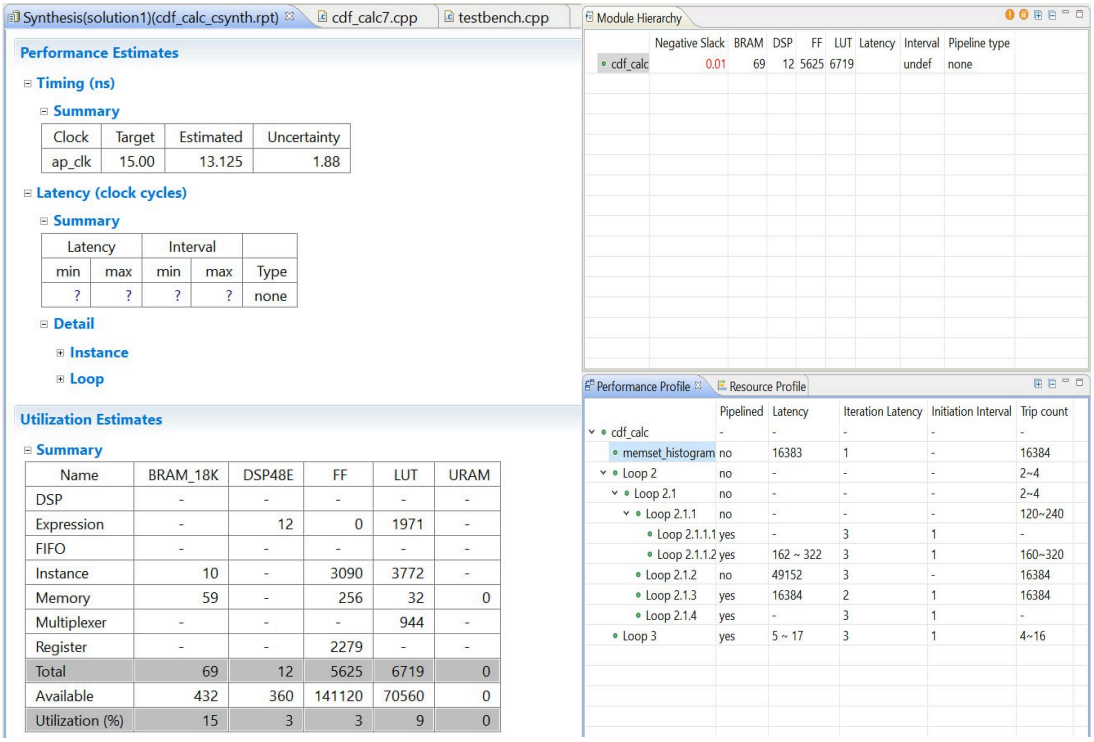


Fig. 11. Resource Utilization and timing analysis of HLS code.

After successful synthesis, the HLS Code can be exported as an IP core and used in the block design in Vivado Integrated Design Environment.

3.4 CDF Calculation IP Core

The CDF Calculation IP Core is the first HLS IP Core to be executed in the design. It first reads the raw pixel array from DDR memory and splits the image into the number of tiles specified by the user, e.g. 4x4, 16 tiles and generates a histogram and a CDF tile by tile. The clip size, minimum and maximum bin for each tile will also be calculated as these values are needed for reducing memory storage space and to reduce the number of DDR memory transactions, thereby reducing the time taken for the core to complete.

3.4.1 Initializing Function and Ports

At the beginning of the CDF HLS Code, the input and output ports and variables must be initialized. In the blue box in Fig. 11, the #pragma lines tell the HLS compiler to connect these variables to an Axi memory-mapped master port connected to the FPGA DDR memory controller IP in the Vivado design. The variables also point to the first address of the array in memory, e.g. reformed_data[0]. The #pragma lines in the red box tell the compiler to bundle these variables as Axilite variables which can be accessed by the ARM processor via the Axilite interface. These ports and values assigned to them are controlled from the user via Xilinx SDK where they will call a function to set the values for e.g. CLIP_LIMIT, WIDTH, HEIGHT etc. Fig. 13 shows the definition of BINS, in this project, the number of bins is $2^{14} = 16384$ for 14-bit IR Images. This means that the maximum intensity or value a pixel can have will be $2^{14} - 1 = 16383$.

```
void cdf_calc (unsigned int* reformed_data, unsigned int* output,
        int* cdf_min_out,
        int* min_bin_out, int* max_bin_out,
        int* clip_size_out,
        int CLIP_LIMIT,
        int WIDTH, int HEIGHT, int NUM_TILE_X,
        int NUM_TILE_Y,
        int TILE_SIZE_X,
        int TILE_SIZE_Y)
{
#pragma HLS INTERFACE m_axi port=reformed_data offset=slave bundle=gmem1
#pragma HLS INTERFACE m_axi port=output offset=slave bundle=gmem1
#pragma HLS INTERFACE m_axi port=cdf_min_out offset=slave bundle=gmem2
#pragma HLS INTERFACE m_axi port=min_bin_out offset=slave bundle=gmem3
#pragma HLS INTERFACE m_axi port=max_bin_out offset=slave bundle=gmem4
#pragma HLS INTERFACE m_axi port=clip_size_out offset=slave bundle=gmem5

#pragma HLS INTERFACE s_axilite port=CLIP_LIMIT bundle=CRTL_BUS
#pragma HLS INTERFACE s_axilite port=WIDTH bundle=CRTL_BUS
#pragma HLS INTERFACE s_axilite port=HEIGHT bundle=CRTL_BUS
#pragma HLS INTERFACE s_axilite port=NUM_TILE_X bundle=CRTL_BUS
#pragma HLS INTERFACE s_axilite port=NUM_TILE_Y bundle=CRTL_BUS
#pragma HLS INTERFACE s_axilite port=TILE_SIZE_X bundle=CRTL_BUS
#pragma HLS INTERFACE s_axilite port=TILE_SIZE_Y bundle=CRTL_BUS
#pragma HLS INTERFACE s_axilite port=return bundle=CRTL_BUS
```

Fig. 12. Initialization of function and ports in HLS CDF core.

```
#include <hls_stream.h>
#include <ap_axi_sdata.h>

#define BINS            16384
```

Fig. 13. BINS initialized in #define.

3.4.2 Histogram Generation

Histogram generation in the CDF core is done tile by tile. The histogram is saved in an array histogram[16384]. On the last pixel of each tile, the code checks if any histogram bin exceeds the clip limit the user sets. The histogram bin that exceeds the clip limit will be clipped to the clip limit value. Once the histograms have been clipped, the CDF for the tile will be generated. The histogram array will reset to zero once the CDF is generated.

It should be noted that the histogram does not spread out across the entire 14-bit range and is only concentrated within a small, restricted bin range. E.g. 9000 to 11000. This helps the design reduce the memory storage used and the number of DDR Memory transactions.

```
if (last_pix_flag)
{
    histogram[old_bin] = accum;

    for (i = 0; i < 16384; i++)
    {

        if (histogram[i] > CLIP_LIMIT)
        {
            new_clip = (histogram[i] - CLIP_LIMIT);
            clip = old_clip + new_clip;
            old_clip = clip;
            clip_size[tile_count_y * NUM_TILE_X + tile_count_x] = clip;
            histogram[i] = CLIP_LIMIT;
        }
    }
    accum = acc_1 = 0;
    old_clip = clip = 0;
```

Fig. 14. Code on clipping the histogram.

3.4.3 CDF Generation

The CDF is generated by cumulatively adding the histogram bin frequency in. The resultant CDF value for that bin will be stored in cdf[i] where i is the bin number.

Fig. 15 shows a snippet of the CDF calculation. The CDF of the tile will be then transmitted to DDR memory ~~first~~.

```
                for (i = 0; i < 16384; i++)
                {
#pragma HLS PIPELINE
                    frequency = histogram[i];
                    cum_freq = frequency + cdf_value;
                    cdf_value = cum_freq;
                    cdf[i] = cdf_value;
```

Fig. 15. Code on calculating and storing CDF values.

3.4.4 Minimum and Maximum Bin

Defining the minimum and maximum bin is crucial to this design, as having these values will store the CDF from the start of its curve; this avoids transmitting the repetition of zeros in the lower bins. Fig. 16 shows how the minimum and maximum bin value is obtained.

```
    if (cdf[i] == 0)
    {
        //zero_count+=1; // max no. is min bin index

        zero_count_1 = zero_count_0 + 1;
        zero_count = zero_count_1;
        zero_count_0 = zero_count_1;
    }

    // Count how many max count values there are

    else if (cdf[i] == (TILE_SIZE_X * TILE_SIZE_Y - clip_size[tile_count_y * NUM_TILE_X + tile_count_x])) // 76800
    {
        //max_count+=1; // 16383 - (max no. - 1) = max bin no. index

        max_count_1 = max_count_0 + 1;
        max_count = max_count_1;
        max_count_0 = max_count_1;
    }
    // Reset Histogram for next tile
    histogram[i] = 0;
    //output_temp_buffer[i] = cdf[i];
}

min_bin[tile_count_y * NUM_TILE_X + tile_count_x] = zero_count;
max_bin[tile_count_y * NUM_TILE_X + tile_count_x] = ((BINS - 1) - (max_count - 1));
int no_of_bins = max_bin[tile_count_y * NUM_TILE_X + tile_count_x] - min_bin[tile_count_y * NUM_TILE_X + tile_count_x] + 1;
```

Fig. 16. Code on obtaining minimum and maximum bin values.

3.4.5 Minimum CDF

Finding the minimum CDF value is essential as it is used in the calculations for transformation function and histogram equalization. The minimum CDF is found by finding the first value after the starting zeros in the CDF array.

```
// Finding smallest value
cdf_min[tile_count_y * NUM_TILE_X + tile_count_x] = cdf[zero_count + 1];
```

Fig. 17. Code on obtaining minimum CDF values.

3.4.5 DDR Memory Transactions

The truncated CDF values, minimum CDF value for each tile, clip size for each tile, minimum and maximum bin for each tile are all sent to DDR memory. The CDF values are sent tile by tile first. The variable bin_count represents the cumulative number of bins present in the tiles and is used for indexing the CDF values in one array. Fig. 18 shows the CDF values being stored in the array “Output” to be sent to DDR memory. The rest of the miscellaneous data is sent to DDR memory only after all the CDF values for each tile have been calculated and transacted to DDR memory as shown in Fig. 19.

```
                for (j = 0; j < no_of_bins; j++)
                {
#pragma HLS PIPELINE
                    //output_temp_buffer[bin_count + j] = cdf[j + zero_count];
                    output[bin_count + j] = cdf[j + zero_count];
                }
```

Fig. 18. Code on transacting CDF values to output array with indexing.

```
    for (j = 0; j < NUM_TILE_X * NUM_TILE_Y; j++)
    {
#pragma HLS loop_tripcount min=4 max=16 avg=4
#pragma HLS PIPELINE
        cdf_min_out[j] = cdf_min[j];
        clip_size_out[j] = clip_size[j];
        min_bin_out[j] = min_bin[j];
        max_bin_out[j] = max_bin[j];
    }
}
```

Fig. 19. Code on transacting the miscellaneous data to DDR memory.

3.5 Bilinear Calculation IP Core

The Bilinear Calculation IP Core starts by initializing the parameters that need to be predefined in the same way as the CDF HLS IP Core, including the width and height of the image, tile size x and y, and the number of tiles in the x and y direction.

After reading the necessary data for the calculations, the code enters loops with conditional statements to check which pixel is being processed. The conditional statements check which tile the current pixel is in and what the x and y coordinates of the pixel are. The code will ascertain which case the current pixel falls under based on the conditions.

After calculating the CLAHE enhanced value for one pixel, the HLS core will output the numerator and denominator values from the equation to the Xilinx external floating-point IP core via Axi4-Stream where floating-point division will be done.

3.5.1 Initializing Function and Ports

In the Bilinear function, the raw pixels and all the CDF HLS IP Core outputs are read into the Bilinear Core. The Axi4-Stream ports are also initialized to have a 64-bit data width and 20-bit tuser width. This is due to some values of the numerator and denominator exceeding 32 bits; hence, a 64-bit data width is used to prevent a bit overflow. A 20-bit tuser width is used as the tuser field is used to transmit the pixel number for the HE HLS block to index the pixel data based on the pixel number. The

#pragma initialization is the same as the CDF HLS block, with an additional block of HLS interface initialization, this tells the compiler to generate these ports as Axi4-Stream ports.

```
typedef ap_axis<64,20,1,1> intsdch;

void bilinear(unsigned int* raw_pixels,
        unsigned int* cdf_in,
        int* cdf_min_in,
        int* min_bin_in,
        int* max_bin_in,
        int* clip_size_in,
        int WIDTH,
        int HEIGHT,
        int NUM_TILE_X,
        int NUM_TILE_Y,
        int TILE_SIZE_X,
        int TILE_SIZE_Y,
        hls::stream<intsdch> &outStream_c1,
        hls::stream<intsdch> &outStream_n1,
        hls::stream<intsdch> &outStream_c2,
        hls::stream<intsdch> &outStream_n2,
        hls::stream<intsdch> &outStream_c3,
        hls::stream<intsdch> &outStream_n3,
        hls::stream<intsdch> &outStream_c4,
        hls::stream<intsdch> &outStream_n4)
```

Fig. 20. Function of the Bilinear HLS Core.

```
#pragma HLS INTERFACE m_axi port=raw_pixels offset=slave bundle=gmem1
#pragma HLS INTERFACE m_axi port=cdf_min_in offset=slave bundle=gmem2
#pragma HLS INTERFACE m_axi port=min_bin_in offset=slave bundle=gmem3
#pragma HLS INTERFACE m_axi port=max_bin_in offset=slave bundle=gmem4
#pragma HLS INTERFACE m_axi port=clip_size_in offset=slave bundle=gmem5
#pragma HLS INTERFACE m_axi port=cdf_in offset=slave bundle=gmem6

#pragma HLS INTERFACE axis port=outStream_c1
#pragma HLS INTERFACE axis port=outStream_n1
#pragma HLS INTERFACE axis port=outStream_c2
#pragma HLS INTERFACE axis port=outStream_n2
#pragma HLS INTERFACE axis port=outStream_c3
#pragma HLS INTERFACE axis port=outStream_n3
#pragma HLS INTERFACE axis port=outStream_c4
#pragma HLS INTERFACE axis port=outStream_n4

#pragma HLS INTERFACE s_axilite port=WIDTH bundle=CRTL_BUS
#pragma HLS INTERFACE s_axilite port=HEIGHT bundle=CRTL_BUS
#pragma HLS INTERFACE s_axilite port=NUM_TILE_X bundle=CRTL_BUS
#pragma HLS INTERFACE s_axilite port=NUM_TILE_Y bundle=CRTL_BUS
#pragma HLS INTERFACE s_axilite port=TILE_SIZE_X bundle=CRTL_BUS
#pragma HLS INTERFACE s_axilite port=TILE_SIZE_Y bundle=CRTL_BUS
#pragma HLS INTERFACE s_axilite port=return bundle=CRTL_BUS
```

Fig. 21. #pragma initialization on Bilinear Core.

3.5.2 Reading in CDF and Miscellaneous Values

The first task the Bilinear core does is to read in the CDF values and the miscellaneous values output from the CDF Core and localize it in an array. The code then splits the CDF values for each tile into a double array to store the values separately, as indexing the CDF double array is an important step in calculating the CLAHE-enhanced pixel values. Each local CDF double array is preset as cdf[16][1000], indicating a maximum of 16 tiles and 1000 CDF values. However, each tile does not practically take up the full storage of 1000 values. As such, there is a need to fix all values to the maximum CDF value for indexes more than the number of bins for standardization.

```
    // Extract the cdf min, min bin, max bin and clip size for each tile first

    for (i = 0; i < NUM_TILE_X * NUM_TILE_Y; i++)
    {
#pragma HLS PIPELINE
        cdf_min[i] = cdf_min_in[i];
        min_bin[i] = min_bin_in[i];
        max_bin[i] = max_bin_in[i];
        clip_size[i] = clip_size_in[i];
    }

    // Extracting the cdf

    for (j = 0; j < NUM_TILE_X * NUM_TILE_Y; j++)
    {
        no_of_bins = max_bin[j] - min_bin[j] + 1;

        for (i = 0; i < no_of_bins; i++)
        {
#pragma HLS PIPELINE
            cdf[j][i] = cdf_in[cum_bin_count + i];

            // For testing
            //cdf_temp[cum_bin_count + i] = cdf_in[cum_bin_count + i];
        }
        index_check = 1000 - no_of_bins;

        for (i = 0; i < index_check; i++)
        {
            // Fix all values to max cdf value for indexes more than no. of bins for
            // standardisation
            cdf[j][no_of_bins + i] = cdf[j][max_bin[j] - min_bin[j]];// check
        }

        cum_bin_count += no_of_bins;
    }
```

Fig. 22. Code for reading in miscellaneous data and CDF values.

3.5.3 Corner Cases

For the corner cases, the conditional statements are such that the extreme corners of the 4 corner tiles will be processed. For example, in the top left tile (tile 0), the Bilinear core will only identify the current pixel as a corner case if the x and y coordinates are less than or equal to half of the tile size of x and y. In Fig. 1, the corner cases are applied to the pixels in the red area. Fig. 23 shows the snippet of the code of the corner case. The variable mode = 1 indicates a corner case. In the formulas applied in code, c1 is the numerator and n1 is the denominator of the transformation function.

```
/*********************************************/
// Corner tiles on LHS of Image
/*********************************************/
if (tile_count_x == 0) // ADDRESSING WHOLE LEFT COLUMN TILE OF THE IMAGE
{
    // TOP LEFT TILE
    if (tile_count_y == 0)
    {
        // top left (corner) tile 0
        if (x <= HALF_TILE_SIZE_X && y <= HALF_TILE_SIZE_Y)
        {
            gen_tile = current_tile;
            mode = 1;
        }

if (mode == 1) // corner
{
    if (index_check_1 < 0)
    {
        c1 = 0;
        n1 = 1;
    }
    else
    {
        c1 = (cdf[gen_tile][pix_buffer[x] - min_bin[gen_tile]] - cdf_min[gen_tile]);

        n1 = ((TILE_SIZE_T - clip_size[gen_tile]) - cdf_min[gen_tile]);
    }
    index_check_1 = 0;
    c2 = c3 = c4 = 0;
    n2 = n3 = n4 = 1;
}
```

Fig. 23. Code of corner case and transformation function calculation.

3.5.4 Edge Cases

The code will identify edge cases if their x and y coordinates fall under the green area in Fig. 1. The two variations of the edge case are shown in Fig. 24, where mode = 2 is linear interpolation in the x direction and mode = 3 is linear interpolation in the y direction. Gen_tile_1 and 2 store the tile number to be processed and pass that value to the general formula in Fig. 25. Fig. 26 shows the code that executes linear interpolation in the y direction. The variables c1 and c2 are the numerators and n1 and n2 are the denominators in the equation. The index checks ensure that part of the numerator gives a non-negative number.

```
}
// top right (edge) tile 0
else if (x > HALF_TILE_SIZE_X && y < HALF_TILE_SIZE_Y)
{
    x1 = HALF_TILE_SIZE_X;
    x2 = HALF_TILE_SIZE_X + TILE_SIZE_X;
    resultant_x1 = x2 - x;
    resultant_x2 = x - x1;

    index_check_1 = pix_buffer[x] - min_bin[current_tile];
    index_check_2 = pix_buffer[x] - min_bin[current_tile + 1];
    gen_tile_1 = current_tile;
    gen_tile_2 = current_tile + 1;
    mode = 2;
}
// Bottom left (edge) tile 0
else if (x < HALF_TILE_SIZE_X && y > HALF_TILE_SIZE_Y)
{
    y1 = HALF_TILE_SIZE_Y + TILE_SIZE_Y;
    y2 = HALF_TILE_SIZE_Y;
    resultant_y1 = y2 - y;
    resultant_y2 = y - y1;

    index_check_1 = pix_buffer[x] - min_bin[current_tile + NUM_TILE_X];
    index_check_2 = pix_buffer[x] - min_bin[current_tile];
    gen_tile_1 = current_tile + NUM_TILE_X;
    gen_tile_2 = current_tile;
    mode = 3;
}
```

Fig. 24. Code of edge cases parameter setting for x and y direction.

```
else if (mode == 2 || mode == 3) // linear interpolation either x or y
{
    // Linear interpolation x direction
    if (mode == 2)
    {
        if (index_check_1 < 0)
        {
            c1 = 0;
        }
        else
        {
            c1 = (cdf[gen_tile_1][pix_buffer[x] - min_bin[gen_tile_1]] - cdf_min[gen_tile_1]) * resultant_x1;
        }

        n1 = ((TILE_SIZE_T - clip_size[gen_tile_1]) - cdf_min[gen_tile_1]) * TILE_SIZE_X;

        if (index_check_2 < 0)
        {
            c2 = 0;
        }
        else
        {
            c2 = (cdf[gen_tile_2][pix_buffer[x] - min_bin[gen_tile_2]] - cdf_min[gen_tile_2]) * resultant_x2;
        }

        n2 = ((TILE_SIZE_T - clip_size[gen_tile_2]) - cdf_min[gen_tile_2]) * TILE_SIZE_X;
    }
```

Fig. 25. Code of linear interpolation in the x direction.

```
    // Linear interpolation y direction
    else
    {
        if (index_check_1 < 0)
        {
            c1 = 0;
        }
        else
        {
            c1 = (cdf[gen_tile_1][pix_buffer[x] - min_bin[gen_tile_1]] - cdf_min[gen_tile_1]) * resultant_y1;
        }
        n1 = ((TILE_SIZE_T - clip_size[gen_tile_1]) - cdf_min[gen_tile_1]) * (-TILE_SIZE_Y);

        if (index_check_2 < 0)
        {
            c2 = 0;
        }
        else
        {
            c2 = (cdf[gen_tile_2][pix_buffer[x] - min_bin[gen_tile_2]] - cdf_min[gen_tile_2]) * resultant_y2;
        }
        n2 = ((TILE_SIZE_T - clip_size[gen_tile_2]) - cdf_min[gen_tile_2]) * (-TILE_SIZE_Y);
    }
    index_check_1 = index_check_2 = 0;
    c3 = c4 = 0;
    n3 = n4 = 1;
}
```

Fig. 26. Code of linear interpolation in the y direction.

3.5.5 Center Cases

The code identifies the center cases by checking if the x and y coordinates are in the center of the image, i.e. neither of the two edge cases. Fig. 27 shows the setting of the variables for the general formula. Mode = 4 indicates a center case. The variables c1 to c4 are the numerators and n1 to n4 are the denominators of the equation.

```
// Bottom right (Bilinear) tile 0
else if (x >= HALF_TILE_SIZE_X && y >= HALF_TILE_SIZE_Y)
{
    x1 = HALF_TILE_SIZE_X;
    x2 = HALF_TILE_SIZE_X + TILE_SIZE_X;
    y1 = HALF_TILE_SIZE_Y + TILE_SIZE_Y;
    y2 = HALF_TILE_SIZE_Y;

    temp_x1 = x2 - x;
    temp_x2 = x - x1;
    temp_y1 = y2 - y;
    temp_y2 = y - y1;

    index_check_1 = pix_buffer[x] - min_bin[current_tile + NUM_TILE_X];
    index_check_2 = pix_buffer[x] - min_bin[current_tile + NUM_TILE_X + 1];
    index_check_3 = pix_buffer[x] - min_bin[current_tile];
    index_check_4 = pix_buffer[x] - min_bin[current_tile + 1];

    gen_tile_1 = current_tile + NUM_TILE_X;
    gen_tile_2 = current_tile + NUM_TILE_X + 1;
    gen_tile_3 = current_tile;
    gen_tile_4 = current_tile + 1;
    mode = 4;
}
```

Fig. 27. Code of Bilinear interpolation parameter settings.

```
else if (mode == 4) // bilinear interpolation
{
    if (index_check_1 < 0){
        c1 = 0;
    }
    else{
        c1 = ((int64_t)cdf[gen_tile_1][pix_buffer[x] - min_bin[gen_tile_1]] - (int64_t)cdf_min[gen_tile_1]) * (int64_t)temp_x1 * (int64_t)temp_y1;
    }

    n1 = (((int64_t)TILE_SIZE_T - (int64_t)clip_size[gen_tile_1]) - (int64_t)cdf_min[gen_tile_1]) * (int64_t)(-TILE_SIZE_T);

    if (index_check_2 < 0){
        c2 = 0;
    }
    else{
        c2 = ((int64_t)cdf[gen_tile_2][pix_buffer[x] - min_bin[gen_tile_2]] - (int64_t)cdf_min[gen_tile_2]) * (int64_t)temp_x2 * (int64_t)temp_y1;
    }

    n2 = (((int64_t)TILE_SIZE_T - (int64_t)clip_size[gen_tile_2]) - (int64_t)cdf_min[gen_tile_2]) * (int64_t)(-TILE_SIZE_T);

    if (index_check_3 < 0){
        c3 = 0;
    }
    else{
        c3 = ((int64_t)cdf[gen_tile_3][pix_buffer[x] - min_bin[gen_tile_3]] - (int64_t)cdf_min[gen_tile_3])
            * (int64_t)temp_x1 * (int64_t)temp_y2;
    }
    n3 = (((int64_t)TILE_SIZE_T - (int64_t)clip_size[gen_tile_3]) - (int64_t)cdf_min[gen_tile_3]) * (int64_t)(-TILE_SIZE_T);

    if (index_check_4 < 0){
        c4 = 0;
    }
    else{
        c4 = ((int64_t)cdf[gen_tile_4][pix_buffer[x] - min_bin[gen_tile_4]]
            - (int64_t)cdf_min[gen_tile_4]) * (int64_t)temp_x2 * (int64_t)temp_y2;
    }
    n4 = (((int64_t)TILE_SIZE_T - (int64_t)clip_size[gen_tile_4]) - (int64_t)cdf_min[gen_tile_4]) * (int64_t)(-TILE_SIZE_T);

    index_check_1 = index_check_2 = index_check_3 = index_check_4 = 0;
}
```

Fig. 28. Code of Bilinear interpolation equations.

3.5.6 Axi4-Stream Output

After a pixel has undergone the calculations from either of the three cases, c1 to n4 will be set to be transmitted via Axi4-stream to the external floating-point IP Core. Fig. 29 shows the assigning of values from c1 to n4. Tuser is set as the current pixel number used for indexing the values on the histogram equalization block. The outstream function pushes out the data onto the Axi4-Stream data bus.

```
    // Prepare and send all values via axi 4 stream
    valOut_c1.data = c1;
    valOut_n1.data = n1;
    valOut_c2.data = c2;
    valOut_n2.data = n2;
    valOut_c3.data = c3;
    valOut_n3.data = n3;
    valOut_c4.data = c4;
    valOut_n4.data = n4;
    valOut_c1.user = valOut_n1.user = valOut_c2.user = valOut_n2.user = valOut_c3.user
            = valOut_n3.user = valOut_c4.user = valOut_n4.user = (start_y + y)* WIDTH + (start_x + x);
    outStream_c1.write(valOut_c1);
    outStream_n1.write(valOut_n1);
    outStream_c2.write(valOut_c2);
    outStream_n2.write(valOut_n2);
    outStream_c3.write(valOut_c3);
    outStream_n3.write(valOut_n3);
    outStream_c4.write(valOut_c4);
    outStream_n4.write(valOut_n4);
}
index += WIDTH;
```

Fig. 29. Code of Axi4-Stream value assigning and transmission.

3.6 Histogram Equalization IP Core

The HE IP Core only needs the image width and height to be initialized by the user as the IP Core is indexing each pixel according to the pixel number. The HE Block receives the four quotients as a 32-bit fixed-point number, where the floating-point IP Core configures it to be 8-bit integer width and 24-bit fractional width. As the sum of the quotients multiplied by 255 gives the resultant CLAHE-enhanced pixel value, The HE Block shifts the decimal point to the left by 16 in the summed 32-bit fixed point number to give a similar value. As the power function in the "math.h" library took 7cc to finish executing, a power function was self-developed to cut down the number of cc it took for this operation. The current implementation of the power function executes at 4cc per pixel, showing improvement in speed over the given function in "math.h". The tuser field holds the current pixel's numbering, making it suitable for indexing. E.g. Pixel 170 = 63, so store output[170] = 63. The resultant output will be sent to DDR memory where the array will be read in Xilinx SDK.

```
typedef ap_axis<32,20,1,1> intsdch;

float multiply_by_power(float value, int power) {
    // Convert the value to an integer representation
    unsigned int* intValue = (unsigned int*)&value;

    // Extract the exponent and significand from the integer representation
    int exponent = ((*intValue >> 23) & 0xFF) - 127; // Extract and unbias the exponent

    // Adjust the exponent by the power
    exponent += power;

    // Re-bias the exponent and ensure it is within range
    if (exponent > 127) {
        exponent = 127; // Clamp exponent to maximum value
    } else if (exponent < -126) {
        return 0.0f; // Underflow: return 0
    }

    // Re-combine the adjusted exponent and the significand
    *intValue = (*intValue & 0x807FFFFF) | ((exponent + 127) << 23);

    // Convert the integer representation back to a float
    return * (float*) intValue;
}
```

Fig. 30. Code of Power function for HE Block.

```
36 void hist_eqn(hls::stream<intsdch> &inStream_d1,
37         hls::stream<intsdch> &inStream_d2,
38         hls::stream<intsdch> &inStream_d3,
39         hls::stream<intsdch> &inStream_d4,
40         int WIDTH,
41         int HEIGHT,
42         int* output_data)
43 {
44 #pragma HLS INTERFACE m_axi port=output_data offset=slave bundle=gmem1
45 #pragma HLS INTERFACE axis port=inStream_d1
46 #pragma HLS INTERFACE axis port=inStream_d2
47 #pragma HLS INTERFACE axis port=inStream_d3
48 #pragma HLS INTERFACE axis port=inStream_d4
49
50 #pragma HLS INTERFACE s_axilite port=WIDTH bundle=CRTL_BUS
51 #pragma HLS INTERFACE s_axilite port=HEIGHT bundle=CRTL_BUS
52 #pragma HLS INTERFACE s_axilite port=return bundle=CRTL_BUS
53
```

Fig. 31. Code of function for HE Block.

```
63     for (i = 0 ; i < total_pix; i++)
64     {
65 #pragma HLS PIPELINE
66
67         intsdch valIn_d1;
68         intsdch valIn_d2;
69         intsdch valIn_d3;
70         intsdch valIn_d4;
71
72         valIn_d1 = inStream_d1.read();
73         valIn_d2 = inStream_d2.read();
74         valIn_d3 = inStream_d3.read();
75         valIn_d4 = inStream_d4.read();
76
77         d1 = valIn_d1.data;
78         d2 = valIn_d2.data;
79         d3 = valIn_d3.data;
80         d4 = valIn_d4.data;
81         index = valIn_d1.user;
82
83         total_d = d1 + d2 + d3 + d4;
84         final_d = multiply_by_power(total_d, -16);
85
86         // Set boundaries in case resultant value > 255 or < 0
87         if (final_d > 255)
88         {
89             final_d = 255;
90         }
91         else if (final_d < 0)
92         {
93             final_d = 0;
94         }
95
96         output_data[index] = final_d;
97     }
98
```

Fig. 32. Code of quotient addition and shifting decimal place for HE Block.

3.7 Vivado IDE

Vivado IDE is a software tool developed by Xilinx for designing, synthesizing, implementing and debugging programmable logic systems like FPGAs. It provides a graphical user interface (GUI) for creating and managing FPGA-based projects. After exporting the HLS IP core into Vivado IDE, the IP cores can be used block design.

3.7.1 CDF Block

Fig. 33 shows the CDF IP Core when exported into the block design in Vivado IDE. On the left of the block it is connected to the clock from the FPGA and a reset function. The bus s_axi_CRTL_BUS was initialized in the HLS Core with the relevant #pragma. This bus is responsible for transacting data inputted by the user on Xilinx SDK to the HLS block directly. On the right of the block, m_axi_gmem1-5 was initialized in the HLS core to transact the CDFs and miscellaneous data to DDR memory where each data has their own bus for parallel transactions. The CDF block will be the first block to be executed in the implementation on Xilinx SDK.

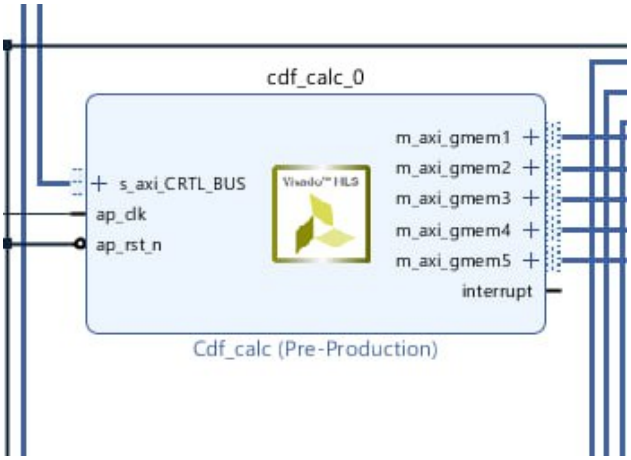


Fig. 33. HLS IP Core of CDF calculation.

3.7.2 Bilinear Block

The left side of the Bilinear block is the same as the CDF block above. On the right side, m_axi_gmem1-5 ports read in the data that was outputted from the CDF block, while the outstream_c1 to n4 output the values of the numerator and denominator of the equations in the 3 cases to the external floating-point IP Core.

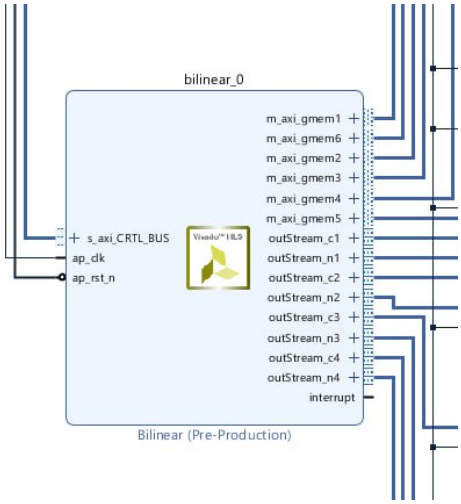


Fig. 34. HLS IP Core of Bilinear calculation block.

3.7.3 Floating-Point IP Core

The floating-point IP Core has three functions in the design. The first is to convert the 64-bit outstream values to a float. The second is to divide c1 by n1, c2 by n2 and so forth, the output will be in a 32-bit float value. Lastly, the floating-point IP core downstream converts the float data type to fixed point, where it will be read by the HE Block. Fig. 35-37 visualizes the floating-point IP core in each function.

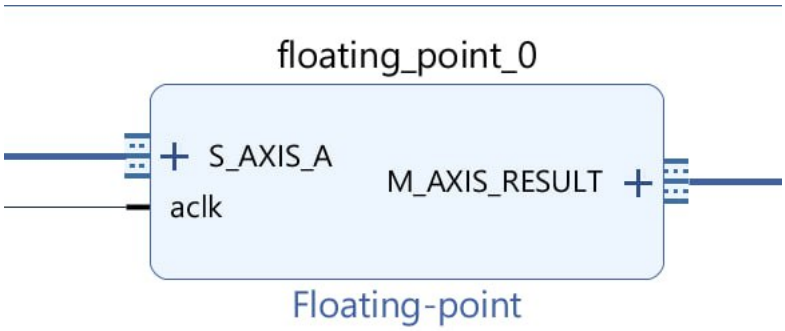


Fig. 35. Fixed-to-Float function in floating-point IP Core block.

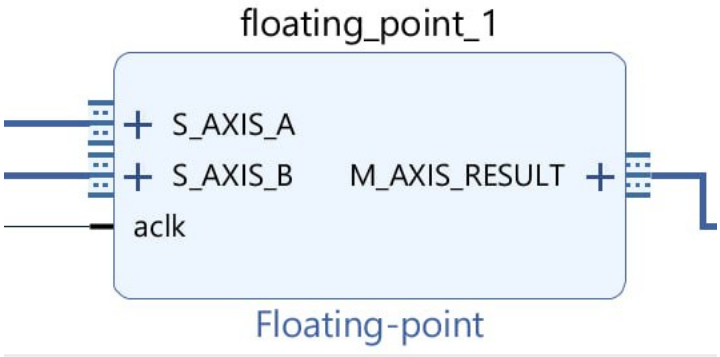


Fig. 36. Divide function in floating-point IP Core block.

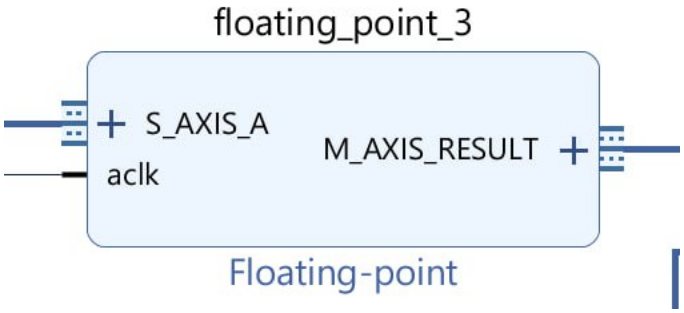


Fig. 37. Float-to-Fixed function in floating-point IP Core block.

3.7.4 Histogram Equalization Block

On the left side of the HE Block, instream_d1-d4 takes in the four quotients outputted from the floating-point IP cores. The s_axi_CRTL_BUS handles the inputs from the user from Xilinx SDK; in this block, the inputs will mainly preset image width and height. The right side port m_axi_gmem1 is the port to transact the final CLAHE-enhanced image array to DDR memory.

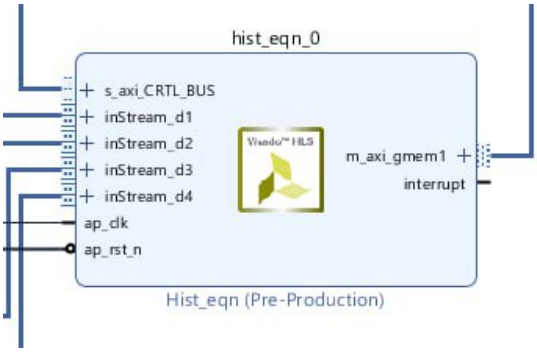


Fig. 38. Histogram Equalization Block.

3.7.5 Overall Block Design

The overall block design can be shown in Fig. 39. There are many floating-point IP core blocks as it caters to the CLAHE design of using bilinear interpolation. The design requires 8 Axi4-Stream outputs from the Bilinear Block and the floating-point IP cores cater for that number.

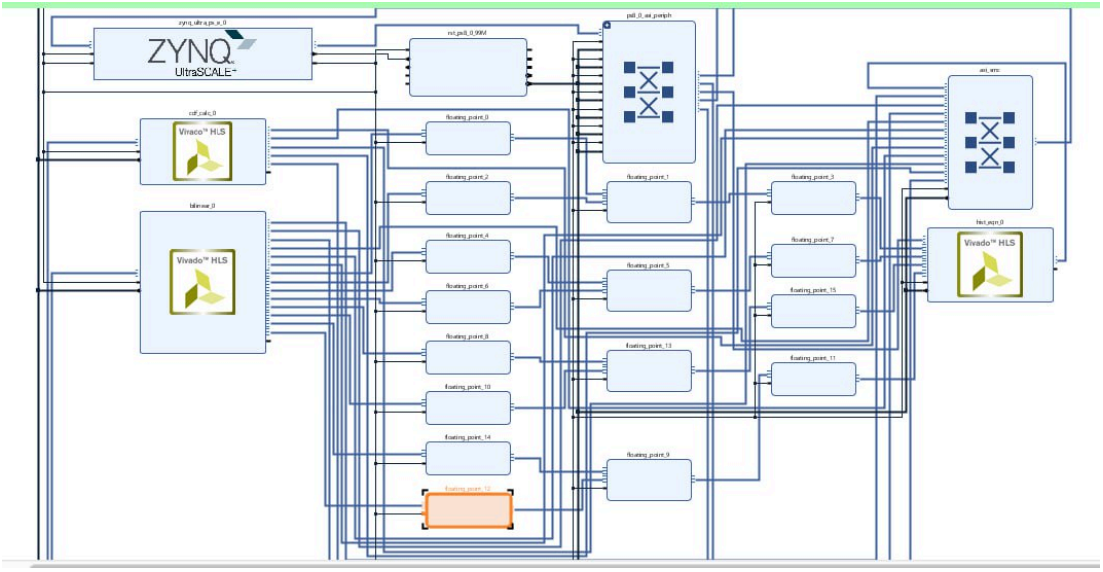


Fig. 39. Overall Block Design of the CLAHE Implementation.

3.8 Xilinx SDK

Xilinx Software Development Kit (SDK) is an IDE that supports software development for embedded systems running on the Xilinx FGPAs and System on Chip (SoC). It provides tools and libraries for embedded processors such as ARM Cortex-A9 Processors. In this project, Xilinx SDK is used to configure the preset parameters for the CLAHE design, take the timing, and calculate the fps of the blocks running using a software timer.

3.8.1 Initializing and Set up

Fig. 40 shows the initializing and set-up of the three HLS blocks and the general initialization of the software timer. In the setup, cache usage is also disabled to prevent the ARM and FPGA from reading/writing old data to the cache instead of directly to the DDR memory. A cache coherent design is not required for this project as for future work, the ARM processor will not need to read/write from DDR memory. This will be handled by a dedicated DMA memory controller in the programmable logic (PL). The initializing sequence checks if the three HLS blocks can be initialized and input the preset parameters into the HLS core.

```
// 16 tile variant
num_tile_x = 4;
num_tile_y = 4;
tile_size_x = 160;
tile_size_y = 120;

//Disable cacheing for ARM to access DDR mem directly
Xil_DCacheDisable();

XTime gbl_time_before_test;
XTime *p_gbl_time_before_test = &gbl_time_before_test;
XTime gbl_time_after_test;
XTime *p_gbl_time_after_test = &gbl_time_after_test;

 //printf("Hello World\n\r");

status = XCdf_calc_Initialize(&cdf_calc, XPAR_CDF_CALC_0_DEVICE_ID);
if (status != XST_SUCCESS)
{
    printf("Cdf_calc initialize fail\n\r");
}
status = XBilinear_Initialize(&bilinear, XPAR_BILINEAR_0_DEVICE_ID);
if (status != XST_SUCCESS)
{
    printf("Bilinear block initialize fail!\n\r");
}
status = XHist_eqn_Initialize(&hist_eqn, XPAR_HIST_EQN_0_DEVICE_ID);
if (status != XST_SUCCESS)
{
    printf("Hist Eqn block init failed\n\r");
}
```

Fig. 40. Initializing three HLS Blocks and Software Timer.

```
// Initialize CDF block variables
XCdf_calc_Set_reformed_data(&cdf_calc, &rawpix[0]);
XCdf_calc_Set_output_r(&cdf_calc, &buffer[0]);
XCdf_calc_Set_cdf_min_out(&cdf_calc, &cdf_min[0]);
XCdf_calc_Set_min_bin_out(&cdf_calc, &min_bin[0]);
XCdf_calc_Set_max_bin_out(&cdf_calc, &max_bin[0]);
XCdf_calc_Set_clip_size_out(&cdf_calc, &clip_size[0]);

XCdf_calc_Set_CLIP_LIMIT(&cdf_calc, 5000);
XCdf_calc_Set_WIDTH(&cdf_calc, 640);
XCdf_calc_Set_HEIGHT(&cdf_calc, 480);
XCdf_calc_Set_NUM_TILE_X(&cdf_calc, num_tile_x); // 2 , 4, 4
XCdf_calc_Set_NUM_TILE_Y(&cdf_calc, num_tile_y); // 2, 3 , 4
XCdf_calc_Set_TILE_SIZE_X(&cdf_calc, tile_size_x); // 320, 160, 160
XCdf_calc_Set_TILE_SIZE_Y(&cdf_calc, tile_size_y); // 240, 160, 120
```

Fig. 41. Setting up parameters for CDF Block.

```
 // Initialize Histogram Equalisation block
XHist_eqn_Set_output_data(&hist_eqn, &bilinear_out[0]);
XHist_eqn_Set_WIDTH(&hist_eqn, 640);
XHist_eqn_Set_HEIGHT(&hist_eqn, 480);

XHist_eqn_Start(&hist_eqn);

XBilinear_Set_raw_pixels(&bilinear, &rawpix[0]);
XBilinear_Set_cdf_in(&bilinear, &buffer[0]);
XBilinear_Set_cdf_min_in(&bilinear, &cdf_min[0]);
XBilinear_Set_min_bin_in(&bilinear, &min_bin[0]);
XBilinear_Set_max_bin_in(&bilinear, &max_bin[0]);
XBilinear_Set_clip_size_in(&bilinear, &clip_size[0]);
XBilinear_Set_WIDTH(&bilinear, 640);
XBilinear_Set_HEIGHT(&bilinear, 480);
XBilinear_Set_NUM_TILE_X(&bilinear, num_tile_x); // 2, 4, 4
XBilinear_Set_NUM_TILE_Y(&bilinear, num_tile_y); // 2, 3, 4
XBilinear_Set_TILE_SIZE_X(&bilinear, tile_size_x); // 320, 160, 160
XBilinear_Set_TILE_SIZE_Y(&bilinear, tile_size_y); // 240, 160, 120
```

Fig. 42. Setting up parameters for HE and Bilinear Block.

3.8.2 Sequence of Starting

The starting sequence of the algorithm begins with the CDF block, the software timer starts when the CDF block is slated to start. The code waits for the CDF block to complete its operations, while the timer continues to run. Only when the block completes its operations, the timer stops and is stored in the variable test_time as a float. The HE block is designed to start operations first to wait for the four quotients from the bilinear block. This would prevent the case where the bilinear block and floating-point IP cores output the quotients while the HE block is not ready to receive the data. A second timer is started when the bilinear block starts its operations. This time, the code waits for both the bilinear and HE block to finish its operations to calculate its timing. This timing is stored in the variable test_time_2 as a float.

```
// Initialize CDF block variables
XCdf_calc_Set_reformed_data(&cdf_calc, &rawpix[0]);
XCdf_calc_Set_output_r(&cdf_calc, &buffer[0]);
XCdf_calc_Set_cdf_min_out(&cdf_calc, &cdf_min[0]);
XCdf_calc_Set_min_bin_out(&cdf_calc, &min_bin[0]);
XCdf_calc_Set_max_bin_out(&cdf_calc, &max_bin[0]);
XCdf_calc_Set_clip_size_out(&cdf_calc, &clip_size[0]);

XCdf_calc_Set_CLIP_LIMIT(&cdf_calc, 5000);
XCdf_calc_Set_WIDTH(&cdf_calc, 640);
XCdf_calc_Set_HEIGHT(&cdf_calc, 480);
XCdf_calc_Set_NUM_TILE_X(&cdf_calc, num_tile_x); // 2 , 4, 4
XCdf_calc_Set_NUM_TILE_Y(&cdf_calc, num_tile_y); // 2, 3 , 4
XCdf_calc_Set_TILE_SIZE_X(&cdf_calc, tile_size_x); // 320, 160, 160
XCdf_calc_Set_TILE_SIZE_Y(&cdf_calc, tile_size_y); // 240, 160, 120

XTime_GetTime(p_gbl_time_before_test); // START SOFTWARE TIMER
XCdf_calc_Start(&cdf_calc);

while (XCdf_calc_IsDone(&cdf_calc) != 1)
{
    // do nothing
}

 XTime_GetTime(p_gbl_time_after_test); // STOP SOFTWARE TIMER

 test_time = (float) (gbl_time_after_test - gbl_time_before_test)/(COUNTS_PER_SECOND);
```

Fig. 43. Starting sequence of CDF block and timing capture.

```
 // Initialize Histogram Equalisation block
XHist_eqn_Set_output_data(&hist_eqn, &bilinear_out[0]);
XHist_eqn_Set_WIDTH(&hist_eqn, 640);
XHist_eqn_Set_HEIGHT(&hist_eqn, 480);

XHist_eqn_Start(&hist_eqn);

XBilinear_Set_raw_pixels(&bilinear, &rawpix[0]);
XBilinear_Set_cdf_in(&bilinear, &buffer[0]);
XBilinear_Set_cdf_min_in(&bilinear, &cdf_min[0]);
XBilinear_Set_min_bin_in(&bilinear, &min_bin[0]);
XBilinear_Set_max_bin_in(&bilinear, &max_bin[0]);
XBilinear_Set_clip_size_in(&bilinear, &clip_size[0]);
XBilinear_Set_WIDTH(&bilinear, 640);
XBilinear_Set_HEIGHT(&bilinear, 480);
XBilinear_Set_NUM_TILE_X(&bilinear, num_tile_x); // 2, 4, 4
XBilinear_Set_NUM_TILE_Y(&bilinear, num_tile_y); // 2, 3, 4
XBilinear_Set_TILE_SIZE_X(&bilinear, tile_size_x); // 320, 160, 160
XBilinear_Set_TILE_SIZE_Y(&bilinear, tile_size_y); // 240, 160, 120

XTime_GetTime(p_gbl_time_before_test); // START SOFTWARE TIMER
XBilinear_Start(&bilinear);

while (XBilinear_IsDone(&bilinear) != 1 && XHist_eqn_IsDone(&hist_eqn) != 1)
{
    // do nothing
}

XTime_GetTime(p_gbl_time_after_test); // STOP SOFTWARE TIMER

test_time_2 = (float) (gbl_time_after_test - gbl_time_before_test)/(COUNTS_PER_SECOND);
```

Fig. 44. Starting sequence of HE and Bilinear Block and timing capture.

3.8.3 Timing Calculation and Printing Values

The timing calculations in Fig. 43 and 44 show test_time being calculated. It takes time to subtract the number of counts before and after the test. The result is divided by the number of counts per second depending on the ARM Processors clock frequency. To find the fps from the result, the inverse of the test_time rounded to 2 decimal places. To print the values of the CLAHE-enhanced image, the printf function is used for the FPGA to display the values on Tera Term in accordance to pixel number. This gives us the same formatting as how we would read and visualize the file in Jupyter Notebook.

```
float fps_cdf = 1/ test_time;
float fps_bilinear = 1/test_time_2;

printf("%d tiles, fps cdf: %.2f, fps bilinear: %.2f\n\r",
        num_tile_x * num_tile_y, fps_cdf, fps_bilinear);
```

Fig. 45. Fps calculation in Xilinx SDK.

```
for (int i = 0; i < 307200; i++)
{
    printf("%d\n\r", bilinear_out[i]);
}
```

Fig. 46. Printing the CLAHE-enhanced pixel values to Tera Term.

3.8.4 Programming FPGA

When the settings have been made, and the code is ready to run, the FPGA should be powered on, and Tera Term should be connected to the laptop with the correct baud rate setting and prepared to record the data. Next, the user programs the FPGA, as shown in Fig. 47 and launches the code on hardware, as shown in Fig. 48. After waiting for the code launch on the FPGA, Tera Term should be receiving the array of CLAHE-enhanced pixels, and logging should be in progress. Once the logging has finished, the user moves to Jupyter Notebook to read the file logged and visualize the output image.

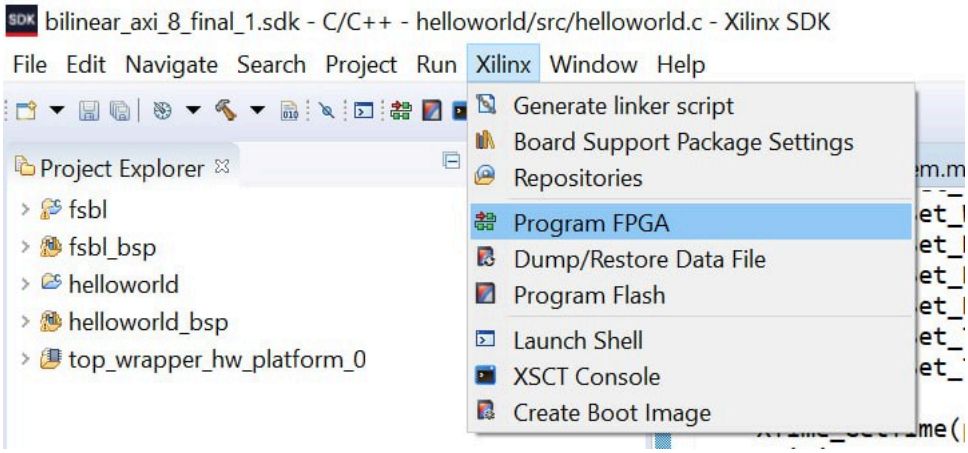


Fig. 47. Steps to Program FPGA.

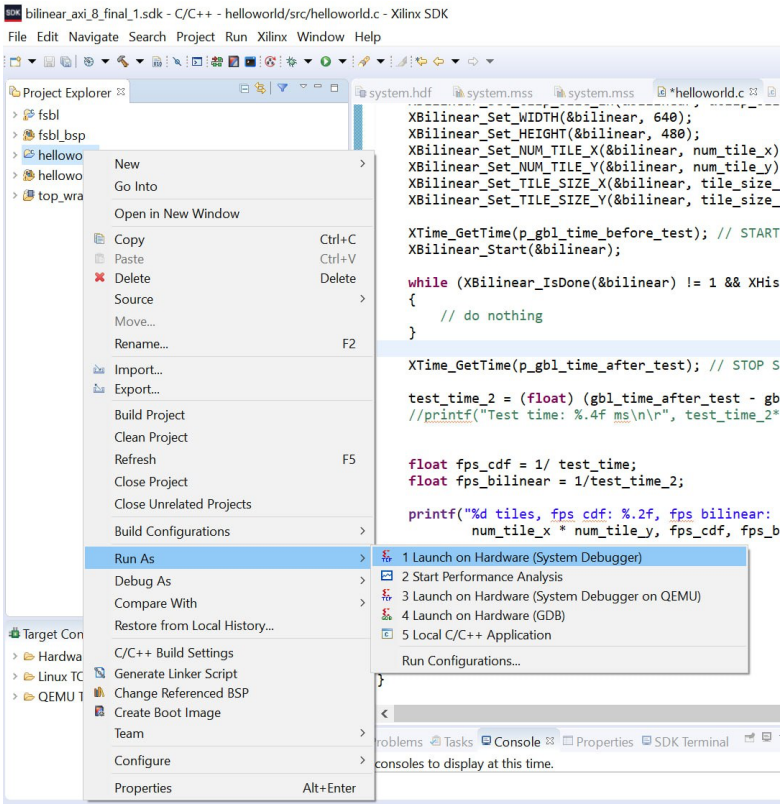


Fig. 48. Steps to launch code on hardware.

CHAPTER 4: RESULTS

4.1 CLAHE-Enhanced Images

Jupyter Notebook is used to read and visualize the images. The given original raw pixel file generates an image, as shown in Fig. 49, where the contrast of the image is poor, and there are a lot of bright spots in the image, making it difficult to see the neighboring areas around those spots. The 4, 12 and 16-tile CLAHE Processes were applied to the same image, and the results are shown in Fig. 50-52. As it can be seen, there is a slight improvement on the 4-tile output compared to the original image where there is better local contrast on the 4-tile output. The top area of the image gets more defined more features can be seen. For the 12-tile variant, there is a huge upgrade in local contrast enhancement compared to the 4-tile variant, where the bottom and middle of the image get more defined. More local features can also be differentiated and seen. The 16-tile output image gives the best contrast-enhanced image as there is improvement in the whole image compared to the 12-tile variant. As such, the first objective in improving the local contrast of images in the CLAHE algorithm has been met. Moreover, the results show that as the number of tiles that split the image increases, the better the local contrast enhancement.

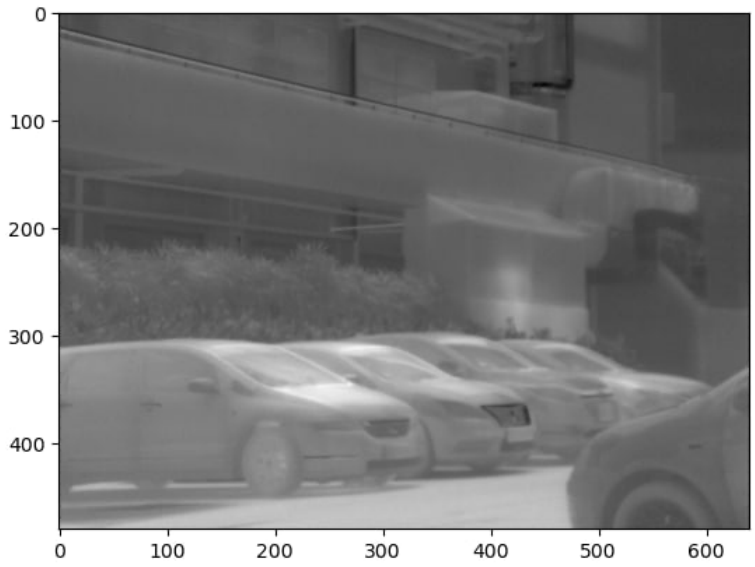


Fig. 49. Original Image from the raw pixels given.

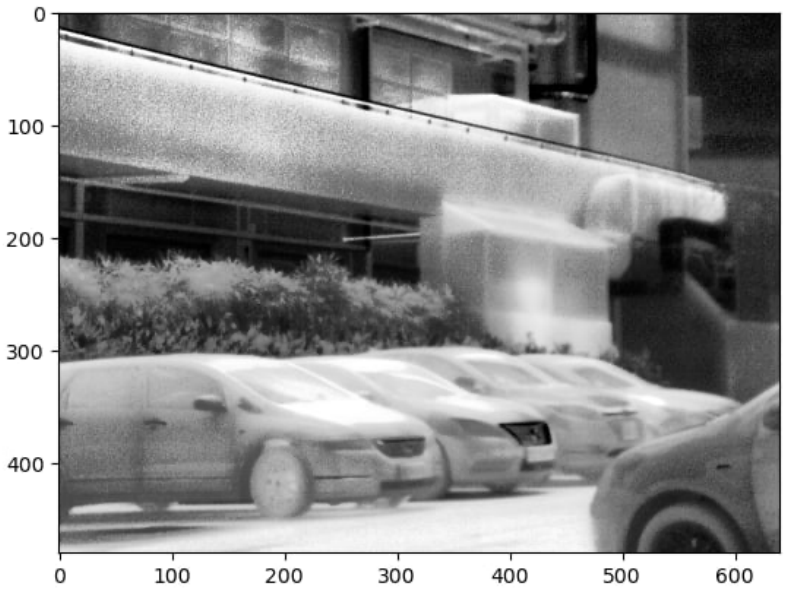


Fig. 50. Image from 4-tile CLAHE variant.

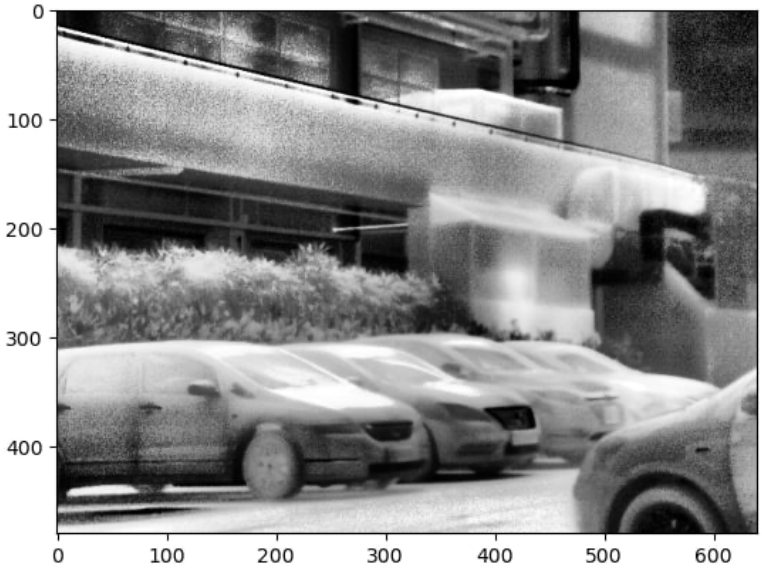


Fig. 51. Image from 12-tile CLAHE variant.

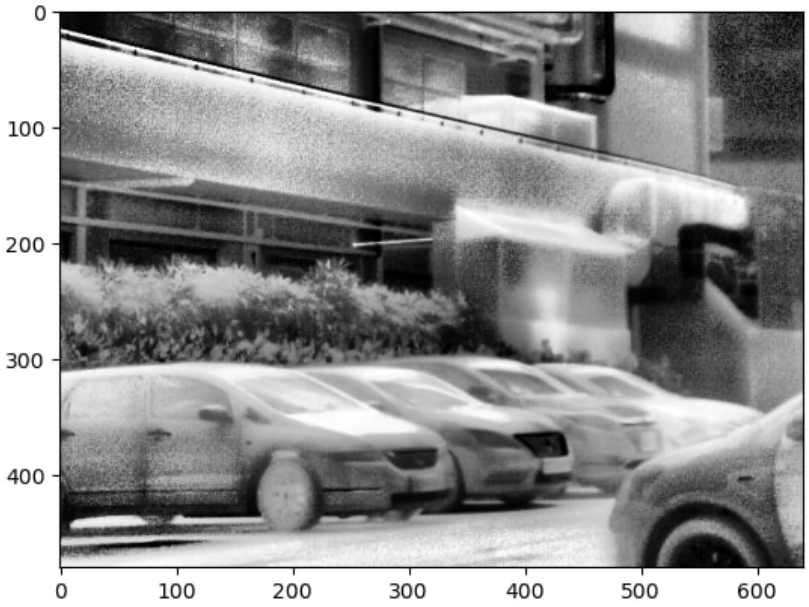


Fig. 52. Image from 16-tile CLAHE variant.

4.2 Throughput on Hardware

The variants for the 4, 12 and 16-tile were run separately, and their throughput results were displayed on Tera Term. It can be seen in Fig. 53 that the CDF, Bilinear and HE block throughput is the highest in the 4-tile variant and the lowest in the 16-tile variant. This is due to the increase in number of histograms and CDFs generated in the CDF Block. In the Bilinear and HE Block, there are more tiles to process, explaining the slight decrease in throughput. Taking the lowest throughput between the two fps, the objective of meeting the throughput requirement of more than 30 fps has been met. This design has even exceeded the throughput requirements. Table 2 compared the throughput results from the different implementations of CLAHE on the FPGA.

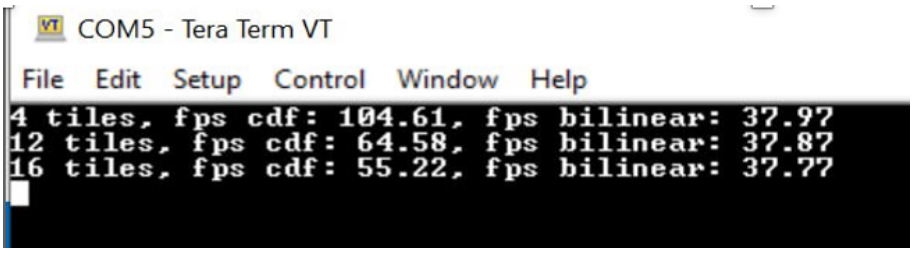


Fig. 53. Throughput results for 4, 12 and 16-tile CLAHE variant for CDF, Bilinear and HE Block.

TABLE 2 BILINEAR THROUGHPUT PERFORMANCE

Implementation	4-tile	12-tile	16-tile
ARM Only	11.24 fps	8.79 fps	7.83 fps
PL with floating-point division in HLS core	18.62 fps	18.23 fps	18.20 fps
PL with floating-point division in RTL	37.97 fps	37.87 fps	37.77 fps

4.3 Resource Utilization

The total resource utilization can be seen in Vivado IDE, which shows how much resource is used for the entire FPGA design. Fig. 54 shows the total amount of resources used broken down for the CLAHE design. Most of the resources are utilized in an acceptable amount, which is less than 50% utilization. The only portion of the resource that is 52% is the Look Up Table (LUT) resource, as it is responsible for implementing combinational logic functions within the FPGA. Furthermore, it is used to handle arithmetic operations and other digital logic functions. As the equations used for the transformation function, histogram equalization and linear/bilinear interpolation are very logic intensive, the FPGA will have to utilize more of the LUT in the implementation. As all the resources are less or close to 50%, it will be considered an acceptable resource usage as utilizing the same FPGA for other algorithms and usages using the remaining resources would be possible. As such, the objective of acceptable resource utilization has been achieved.

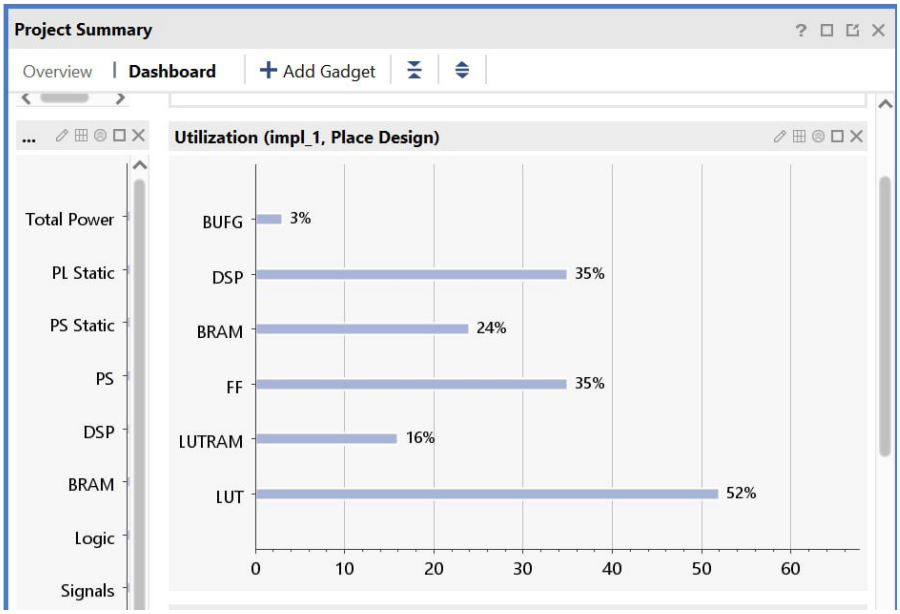


Fig. 54. Resource Utilization for entire CLAHE hardware design.

CHAPTER 5: CHALLENGES FACED

5.1 CLAHE Algorithm Implementation

One of the first challenges was implementing the CLAHE Algorithm code and getting the correct output. Before implementing the code on HLS, a trial test of the algorithm for the code was done on Xilinx SDK to quickly and efficiently test if the approach of implementing the three cases was correct. At the beginning, some of the image pixels were superimposed on each other and had very bad transitions among the tiles, overall making it have a convoluted look which was wrong. Fig. 55 shows one example of a failed implementation run. Due to this, there was a lot of troubleshooting of the conditional statements and equations of the three cases. In the end, it was found that there were errors in both the conditional statements in determining which case the current pixel is in, and the equations of the transformation function and bilinear interpolation. After rectifying the errors, the 4-tile CLAHE-enhanced image displayed was as in Fig. 50.

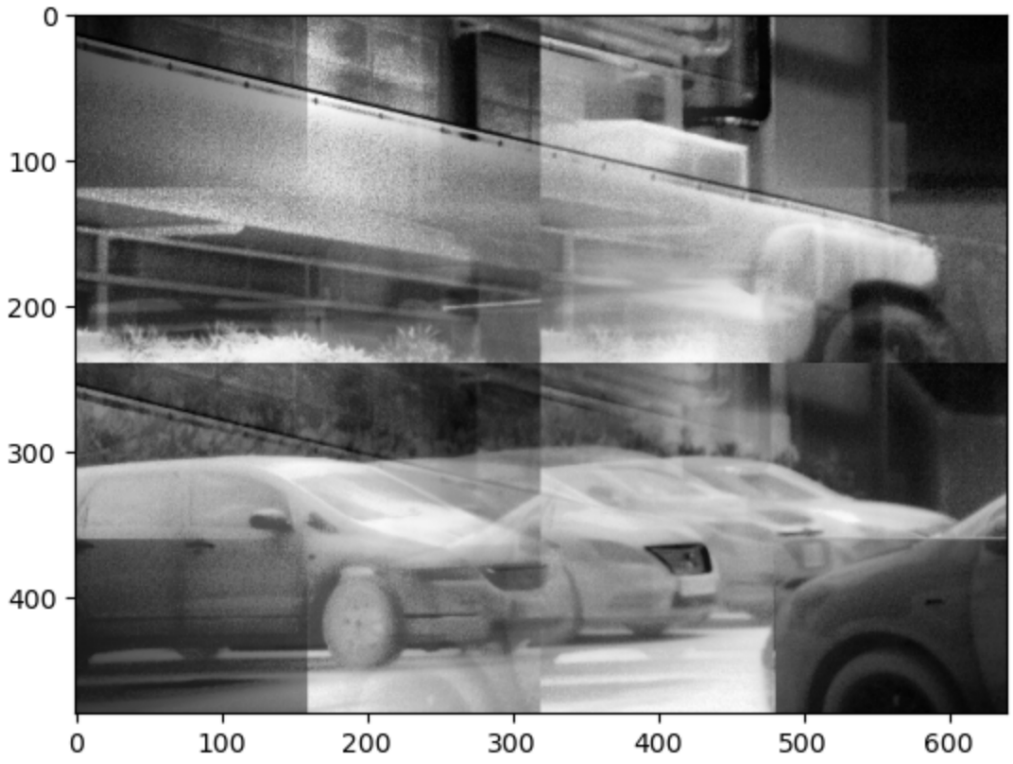


Fig. 55. Erroneous 4-tile CLAHE-enhanced image on Xilinx SDK.

5.2 Meeting Throughput Requirements

In an older Bilinear HLS design, the HE Block was incorporated in it as well, as such, floating point division had to be done in the Bilinear block. Although this made the design simpler, as there was a total of 2 HLS blocks instead of 3, floating point division done in the HLS core took 7cc to execute, severely affecting the throughput of the bilinear block. Fig. 56 shows the fps calculated from the old Bilinear HLS Core design, which only hit about 18 fps. Hence, the solution which involved the external floating point IP Core was used as the old design could not hit the 30-fps requirement.

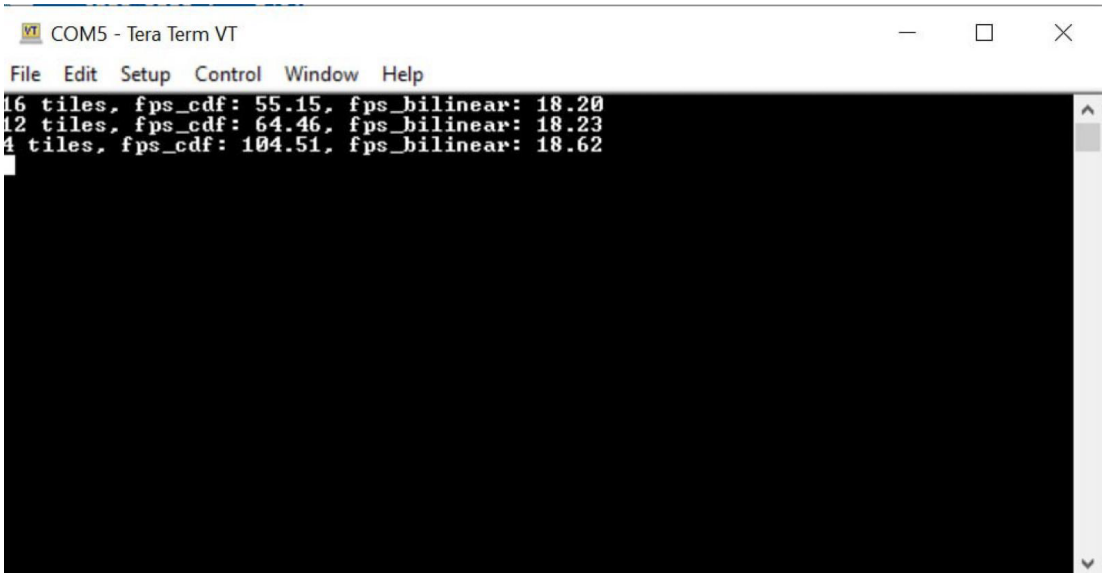


Fig. 56. Fps of Old Bilinear HLS Core Design.

5.3 Workings of Floating-Point IP Core

To implement the external floating-point IP core, knowledge of Axi4-Stream and the workings of the IP Core were needed. Test projects had to be done on the side to test the Axi4-Stream and floating-point IP core separately. Once these test projects worked, there was a greater understanding of how the floating-point IP core and Axi4-Stream work. Knowledge of fixed-point numbers and the IEEE 754 format was needed to fully understand the workings of the fixed-point quotient output from the floating-point IP core.

5.4 Resource Management in Bilinear HLS Core

In the previous version of the Bilinear HLS Core, issues arose over the overutilization of the Digital Signal Processor (DSP) resource. The DSP is designed to execute mathematical operations. In the previous version of the Bilinear HLS code, the equations of the three cases were inside every conditional statement. This resulted in the overutilization of the DSP resources as the calculations were done in parallel ~~processing~~. To rectify this over-utilization issue, the three case equations were generalized and were to be executed only at the end of the loop. This means

only the parameters that needed to be fed into the generalized equations were set in these conditional statements as shown in Fig. 26-28.

CHAPTER 6: FUTURE WORK

6.1 Real-Time Operations

The future of this project should focus on the real-time application of generating a CLAHE-enhanced image with an IR camera connected to the FPGA. The implementation would consist of the CDF and Bilinear blocks running concurrently. The CDF for one tile would be outputted to DDR memory and to be read by the bilinear block. While the bilinear and HE block are computing the resultant CLAHE pixels of that tile, the CDF block will compute the CDF of the next tile. This concurrent process will be feasible for real-time application. The final throughput of the design would also depend on the slowest fps, which in this design is the bilinear and HE block.

CHAPTER 7: CONCLUSION

7.1 Requirements

Overall, the objectives of the project have all been met. Firstly, this includes the contrast enhancement of 14-bit IR monochrome images in the 4, 12 and 16-tile variants of the CLAHE Algorithm. Secondly, the throughput requirements have been met as the Bilinear and HE Block fps averages about 37 fps, hitting above the 30-fps requirement. Lastly, the requirement of resource utilization has been met as most of the resources are under 50% utilized, this enables the FPGA to host another algorithm for other uses, this increases versatility in the overall FPGA design and system.

7.2 Findings Based on Results

The results obtained show that an increase in the number of tiles that split the image increases the local contrast of the image but reduces the throughput of the CDF and Bilinear and HE block. This is because more histograms and CDFs are generated in the CDF block. Similarly, more tiles will be processed and computed for the Bilinear and HE Block. The CDF throughput is greatly affected by the increase in number of tiles in the design. However, similar fps results are obtained for the Bilinear/HE Block for the 4, 12 and 16-tile variants, the difference is less than 0.1 fps when increasing the number of tiles. As the overall throughput of the design depends on the block with the lowest throughput, there are not many implications with the significant difference in fps for the CDF block as the slowest fps on the CDF block is about 55 fps, which is still faster than the fastest fps of the bilinear block. Therefore, the decrease in fps for the CDF block does not make a difference in the overall throughput of the design.

7.3 Scalable Implementation

This approach to implementing the CLAHE Algorithm on the FPGA overcomes the Xilinx HLS implementation limitation of the input being restricted to an 8-bit value. This limitation exists as the CDFs are stored internally in the BRAMs of the FPGA. As CDF memory storage increases exponentially with pixel width, 2^n where n is the pixel width, the Xilinx HLS implementation is limited to processing 8-bit inputs only. This project's approach solves this scalability issue by only storing the portion of CDF where the CDF curve starts and ends. This can be done as the IR image pixel histogram spread is only within a small range of values and does not take the entire 14-bit range. In addition, the external DDR memory is used to store these CDF values. This overcomes the limitation of limited internal FPGA memory storage.

CHAPTER 8: REFLECTION

8.1 Technical Skills Learnt

Throughout the FYP, many technical skills and best practices have been acquired, many of these will are translatable skillsets used in the industry.

8.1.1 Software Debugging

In the initial phases of the project, I tried my hand at implementing the CLAHE Algorithm entirely on the ARM Processor to ensure that the algorithm code was working, to begin with. Even though this was the first step to trial the implementation, it also was one of the toughest as not only was the code over 1000 lines long, there were also many bugs in the code to troubleshoot. Through diligently looking through the code piece by piece, I was able to appreciate and understand the algorithm even more. In addition, I have learnt best practices on commenting, documentation and breakpoints to understand where the issue on the code might lie.

8.1.2 Hardware Debugging

On Vivado, there is a hardware debugger named Chipscope and it is used to monitor signals of the users choice he wants to monitor. Learning how to utilize chipscope was extremely important in this project as it helped to troubleshoot and identify causes of an erroneous design. One example was that the Axi4 Stream was not outputting TVALID, which posed a problem as the floating-point IP Core did not take in the values for division. Chipscope helped to narrow down the problem and is a tool an FPGA designer needs to know.

8.1.3 Improvement in C Language Coding

As most of the CLAHE algorithm was implemented on Vivado HLS, the algorithm was coded in C language. As such, I had to be competent in the C coding language. Generally, my skills in the language improved over time as the project progressed and I can confidently say my C language skills have improved tremendously. As

most Embedded systems and Microcontrollers are coded in C language. This project has improved my capability to code on other hardware platforms.

8.1.4 Improvement in Python Language Coding

As I had to use Jupyter Notebook to visualize my images, and to make the given dataset into an array form, I had to learn a portion of Python Language to be able to do these tasks. After this project, I have improved and acquired the skillset of coding in Python well.

8.1.5 Importance of Pipelining

In Vivado HLS, I learnt that pipelining the code was extremely important and understood how it worked. With pipelining, the FPGA can run lines of code in parallel, increasing the throughput and decreasing the latency of the implemented design. This skill of learning how to pipeline loops, in general, is a very valuable skill in FPGA programming as FPGA designs have timing requirements to hit.

8.1.6 Knowledge on FPGA Resource Assignment

Coding in the Vivado HLS environment has made me aware of the different resources utilized for the algorithm. After having the issue with the overutilization of the DSP, I experimented and understood how the FPGA assigns its resources based on the C code in HLS. With this understanding, it is possible to reduce the resource utilization and will be useful in future FPGA HLS designs.

8.1.7 Operation Latency of Different Data Types

Due to the issue of slow floating-point division in the HLS Core, some experimenting was done to check the latency and iteration interval for division of an float, integer and fixed point. Through this experiment, I learnt that floating-point division in the HLS core takes up a lot of latency and iteration interval in the design and hence decreased the throughput. This was an important factor to decide to utilize

the Xilinx external floating-point IP core, which performs the division operation much faster.

8.2 Personal Skills Learnt

8.2.1 Communication

Throughout the FYP, I was in close communication with my NTU supervisor and DSO supervisor to update them on my progress and notify them when I was stuck on an issue. With this close communication, my DSO supervisor would always know when to help and when to let me figure things out on my own so as to not cut short my learning curve.

8.2.2 Perseverance

In the troubleshooting phase of the project, a lot of perseverance was needed as about 30 minutes is needed each time the FPGA can successfully compile and implement the HLS and ARM code. As such, when troubleshooting the design, the trial and error took a hefty amount of time and required perseverance and diligence to check and test the design in every run.

8.2.3 Time Management

Due to the hefty amount of time needed to troubleshoot and implement the algorithm on the FPGA, I struggled to balance work from my other modules with this project. However, once I started to plan my time and prioritize my tasks, I was able to complete this project while staying on task with assignments from other modules.

8.2.4 Responsibility

I felt that I also grew in responsibility as the FGPA board, and its relevant cables were loaned to me. As the hardware is DSO property, I made a point to take extra care of the hardware and ensured that it was kept in good condition. I believe this spirit of taking care of the equipment fosters best practices in the work place in the future.

REFERENCES

[1] A. M. Reza, “Realization of the contrast limited adaptive histogram equalization (CLAHE) for real-time image enhancement,” *J. VLSI Signal Process. Syst. Signal Image Video Technol.*, vol. 38, pp. 35–44, 2004.

[2] Advanced Micro Devices, “What is an FPGA? Field Programmable Gate Array,” AMD. Accessed: Feb. 20, 2024. [Online]. Available: https://www.xilinx.com/products/silicon-devices/fpga/what-is-an-fpga.html

[3] F. Zakaria and H. Emran, “Stress Fracture Detection using Adaptive Histogram Equalization (Ahe) and Contrast Limited Adaptive Histogram Equalization (Clahe): A Comparative Study,” *Health Scope*, vol. 3, no. 1, pp. 47–50, 2020.

[4] T. Kryjak, K. Blachut, H. Szolc, and M. Wasala, “Real-Time CLAHE Algorithm Implementation in SoC FPGA Device for 4K UHD Video Stream,” *Electronics*, vol. 11, no. 14, p. 2248, 2022.

[5] Advanced Micro Devices, “Box Filter • Vitis Libraries • Reader • AMD Technical Information Portal.” Accessed: Mar. 26, 2024. [Online]. Available: https://docs.amd.com/r/en-US/Vitis_Libraries/vision/api-reference.html_2_17

[6] Advanced Micro Devices, “How AXI4-Stream Works • Vitis High-Level Synthesis User Guide (UG1399) • Reader • AMD Technical Information Portal.” Accessed: Mar. 26, 2024. [Online]. Available: https://docs.amd.com/r/en-US/ug1399-vitis-hls/How-AXI4-Stream-Works

Project No: B2100-231

ACKNOWLEDGING/DECLARING THE USE OF GAI

Please refer to NTU's Current Policy & Guidelines on the Use of Generative AI available in NTUlearn home page and the link: https://entuedu.sharepoint.com/sites/Student/dept/ctlp/SitePages/Exploring-the-Impact-of-Generative-Artificial-Intelligence-(GAI)-Tools-on-Education.aspx

1. Complete the following declaration if applicable.
2. Create a Paper Trail to document the input prompt, output obtained, and how you have used it.

I Yoong Khai Jie, Nathaniel, nyoong002@e.ntu.edu.sg, honestly and sincerely make the following declaration in relation to the following course submission.

1. Name of course: Final Year Project
2. Course Code: EE4080
3. Instructor: Assoc Prof Radhakrishnan K
4. Title of Assignment/Project Submission: CLAHE HLS Implementation on Zynq SoC FPGA

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only	☐
ii. Used GAI as permitted to assist in generating a first text only.	☐
And/or	
iii. Used GAI to refine syntax and grammar for correct language submission only.	☐
Or	
iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.	☑

I also declare that I have:

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that
b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,
c. Apart from the foregoing notices, the submission is wholly my own work.

Yoong Khai Jie, Nathaniel
Student Name & Signature

15 May 2024
Date